Transformers

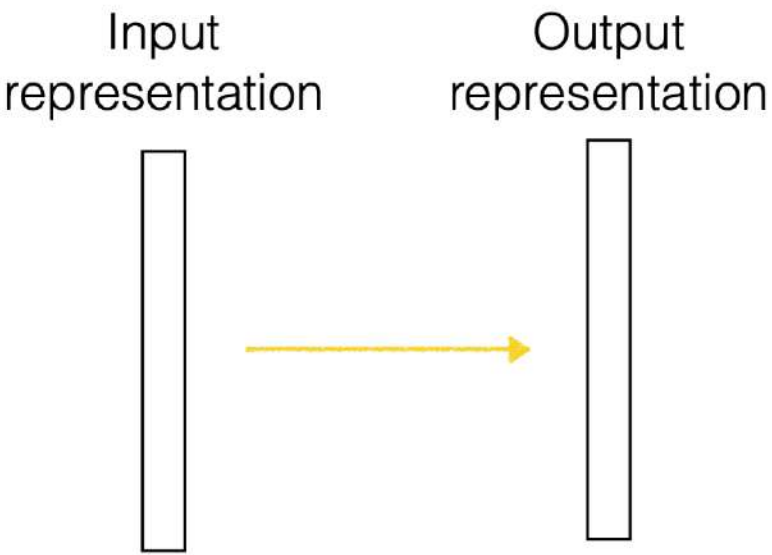
Recap and Visual Transformers

Caltech EE/CMS/CNS 148a - Winter 2026
Michael Hobley

Slides adapted from Sara Beery, MIT

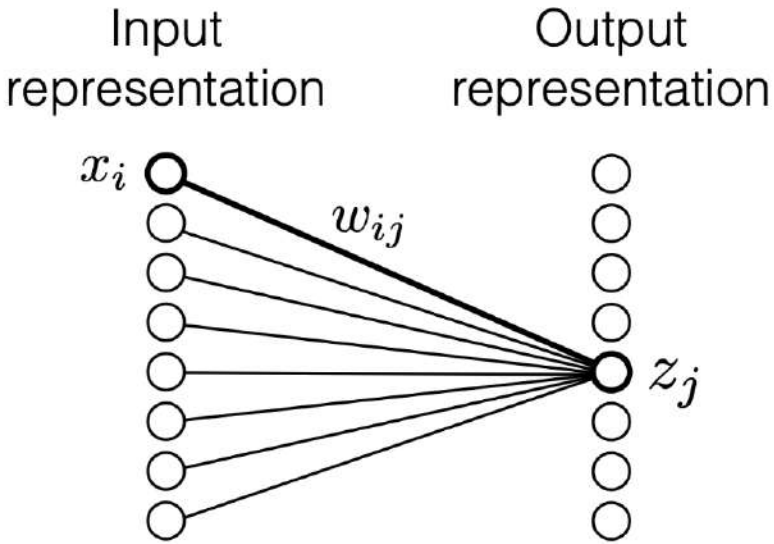
What math happens inside a neural net?

Computation in a neural net



Computation in a neural net

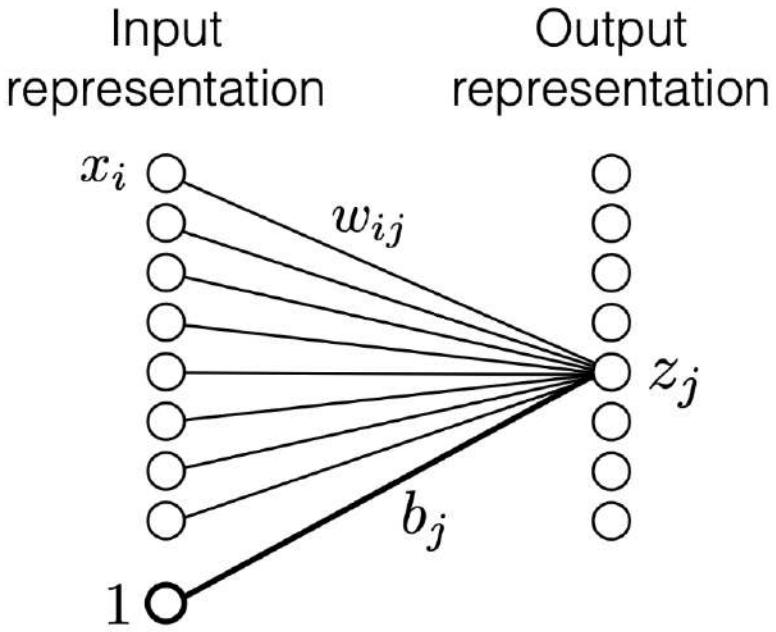
Linear layer



$$z_j = \sum_i w_{ij} x_i$$

Computation in a neural net

Linear layer



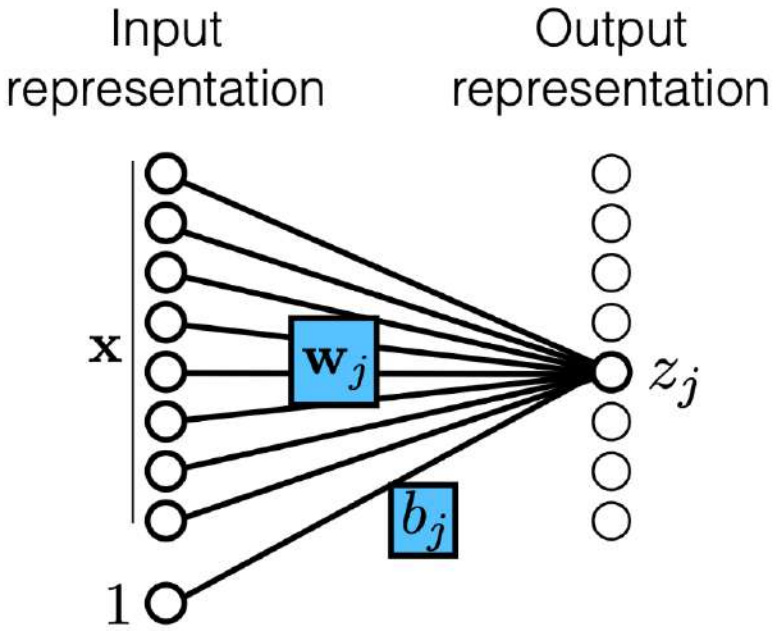
$$z_j = \sum_i w_{ij} x_i + b_j$$

weights

bias

Computation in a neural net

Linear layer



weights

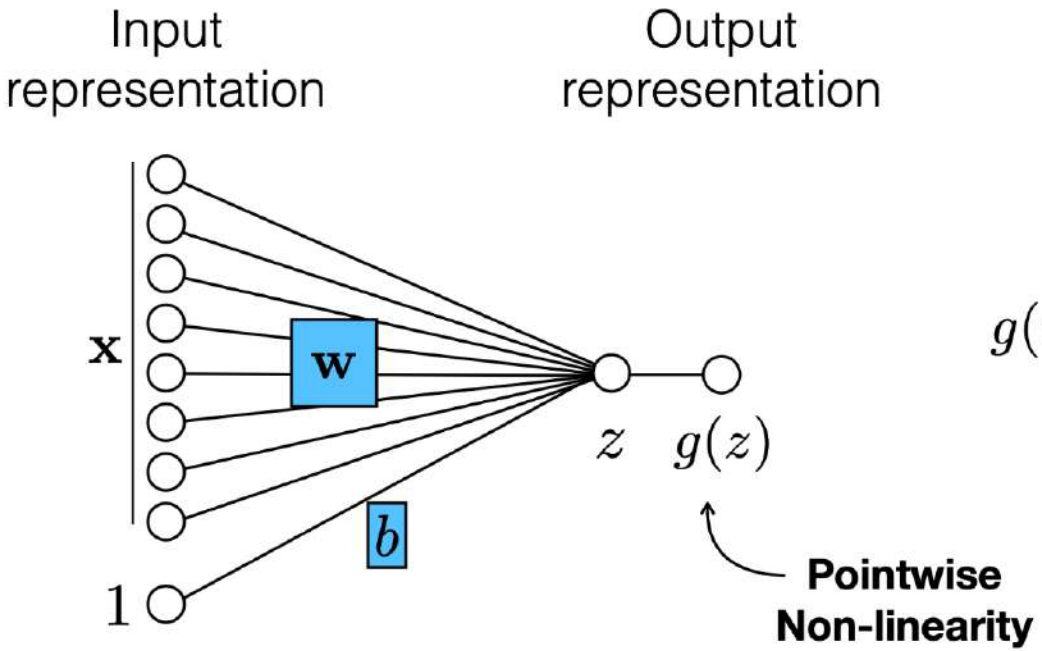
$$z_j = \mathbf{x}^T \mathbf{w}_j + b_j$$

bias

$$\theta = \{\mathbf{W}, \mathbf{b}\}$$

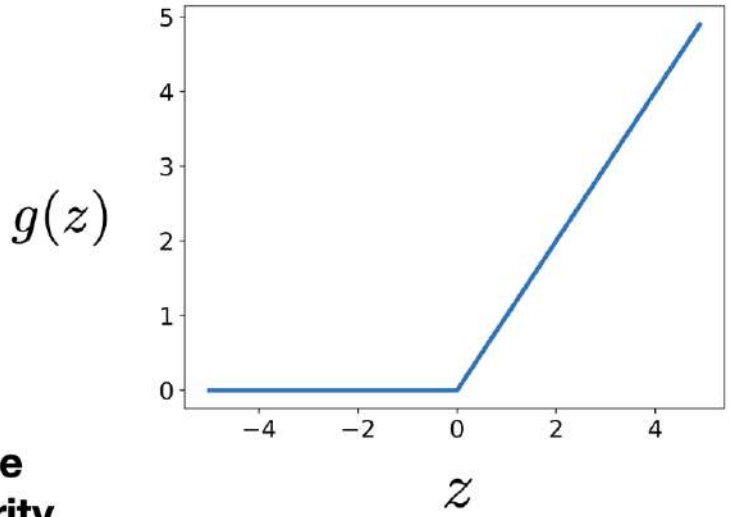
parameters of the model

Single-layer neural net

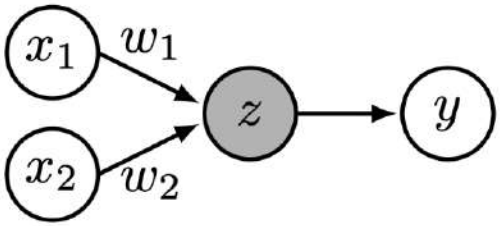


Rectified linear unit (ReLU)

$$g(z) = \max(0, z)$$

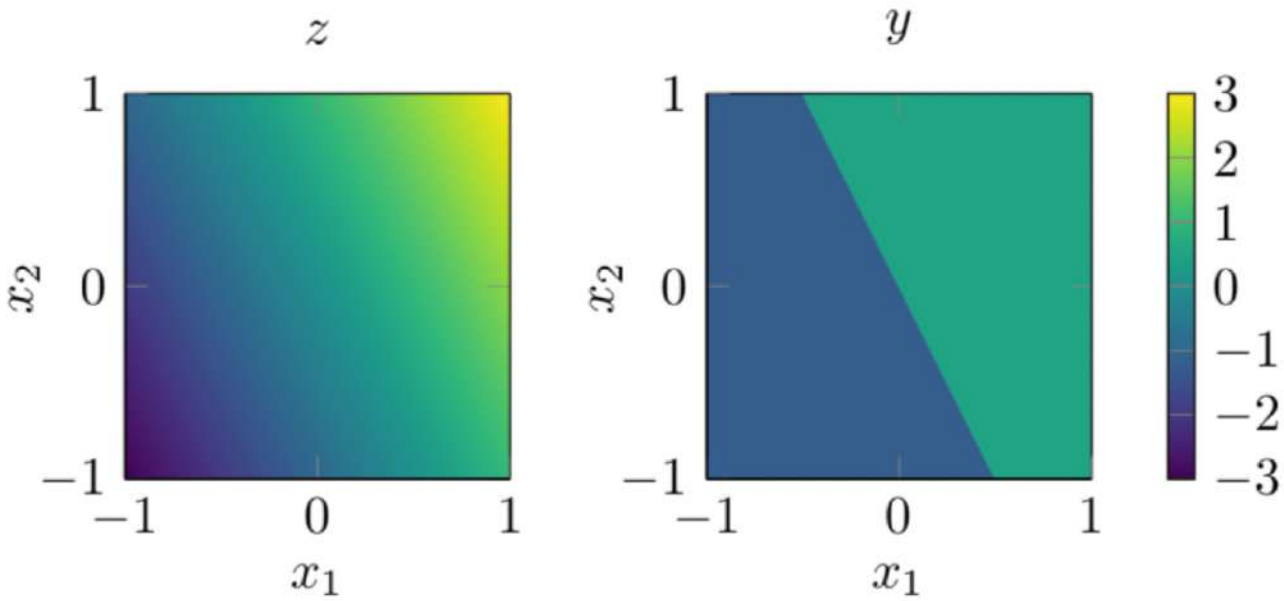


Example: linear classification with a perceptron

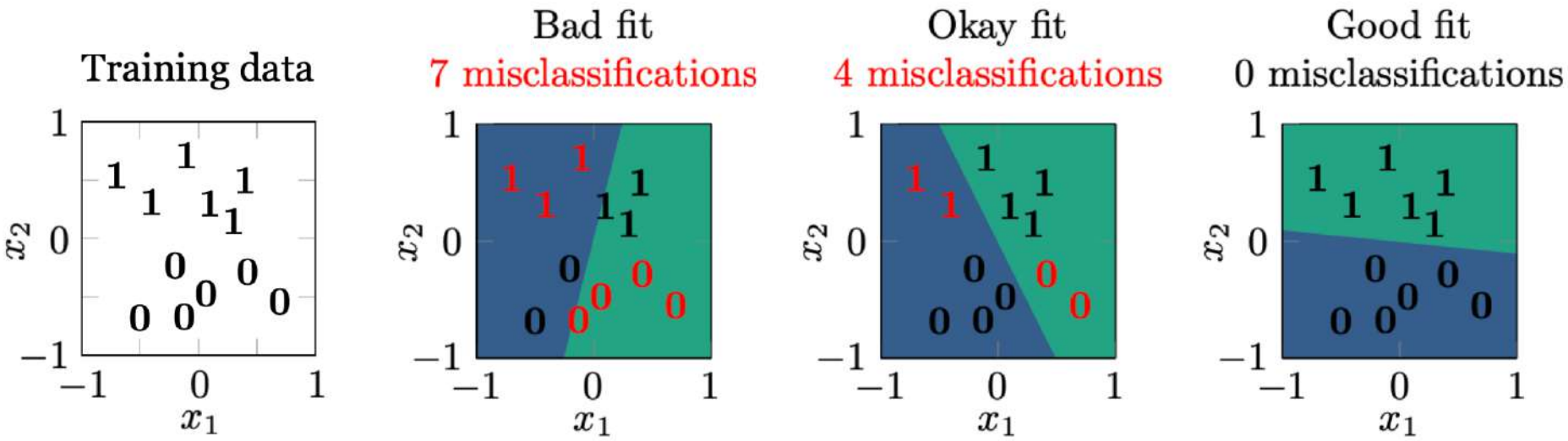


$$z = \mathbf{x}^T \mathbf{w} + b$$

$$y = g(z)$$



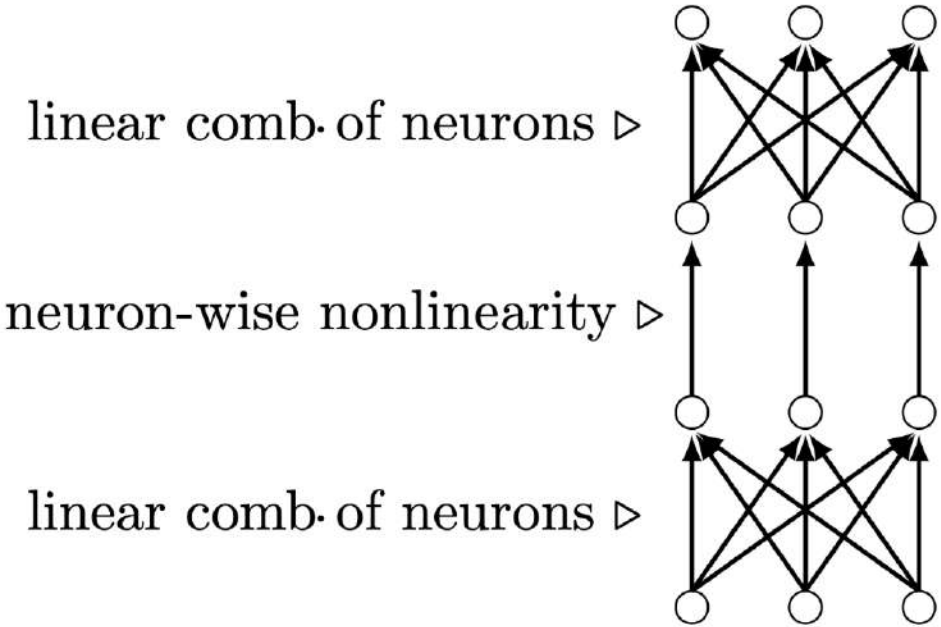
One layer neural net (perceptron) can perform linear classification!



$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w},b} \sum_{i=1}^{N} \mathcal{L}(g(z^{(i)}), y^{(i)})$$

Why do we need different architectures?

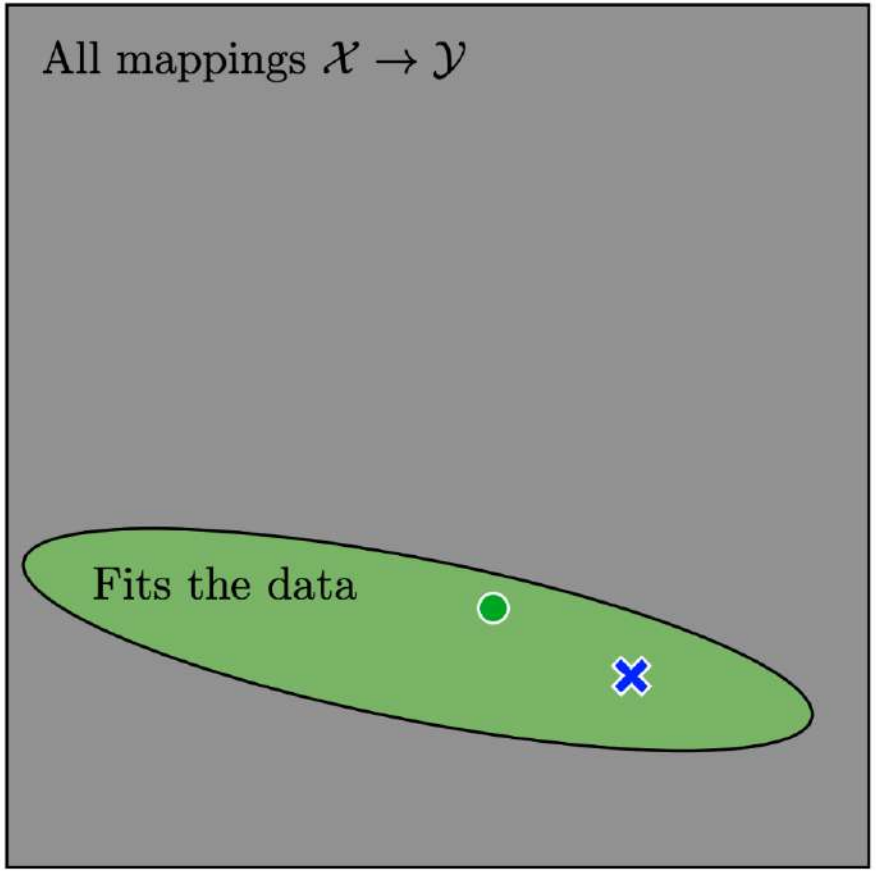
Multilayer Dense Neural Nets



+ Universal
+ Simple (elegant theory)
+ Embarassingly parallel

- Weak inductive biases
- Sample inefficient / data hungry
- Dense (fully-connected) linear layers take a lot of compute

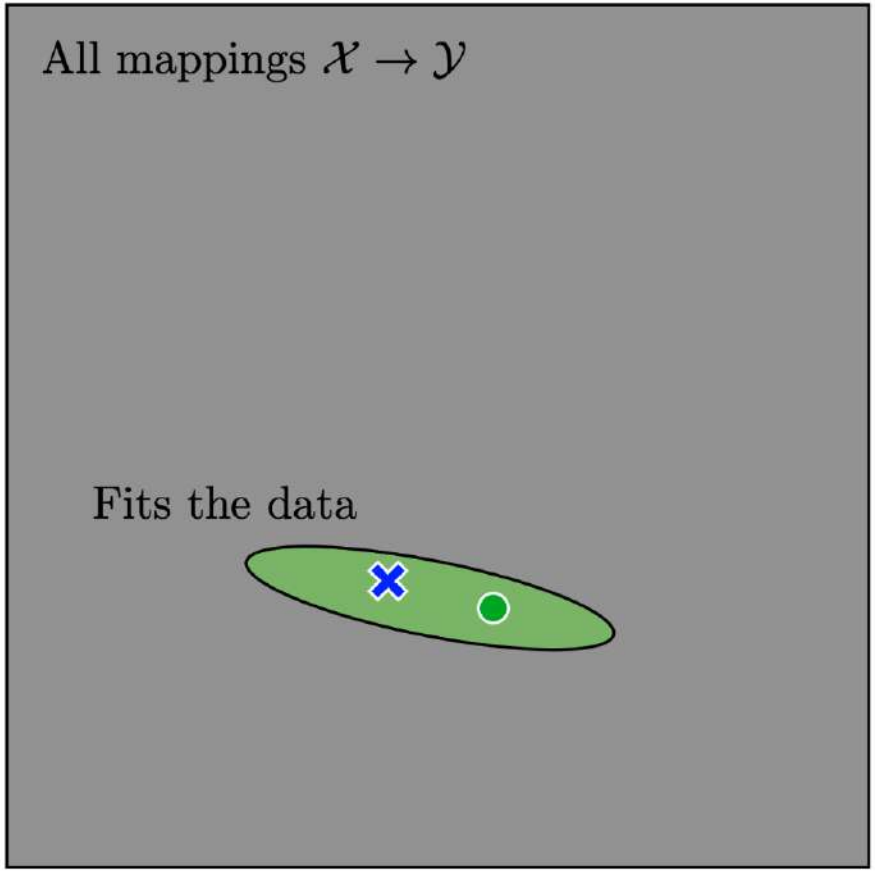
Why use other architectures?



- ● True solution
- ✖ Learned solution

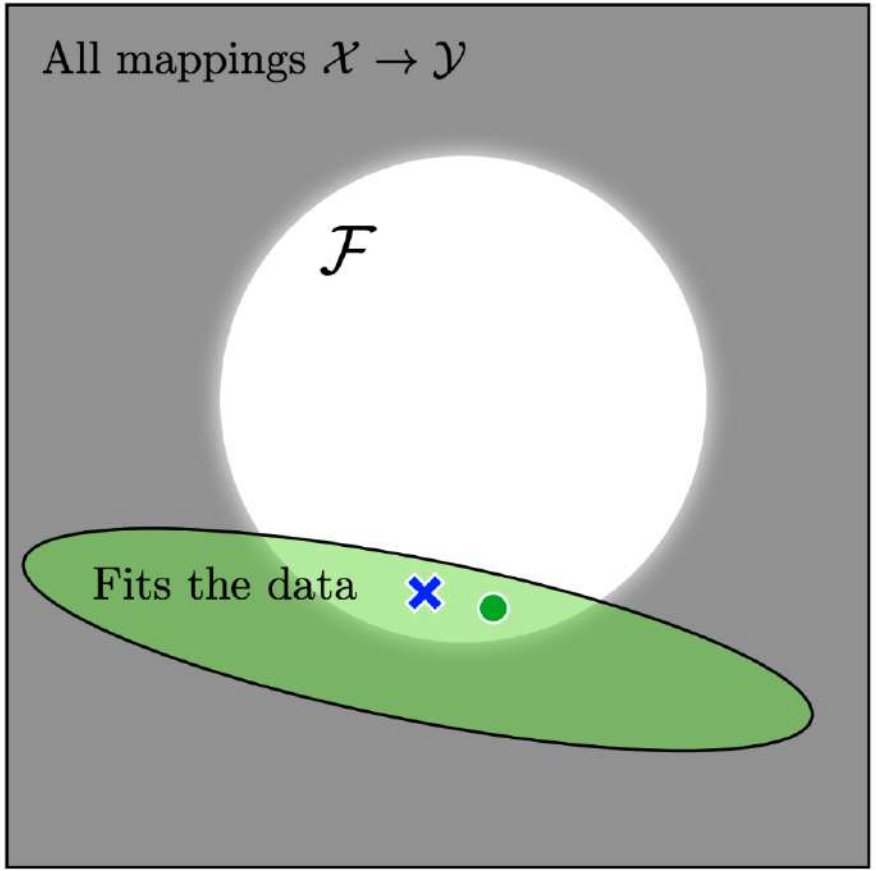
What if we add more data?

Why use other architectures?



- True solution
- Learned solution

Why use other architectures?



$\mathcal{F}$ – Hypothesis space

● True solution

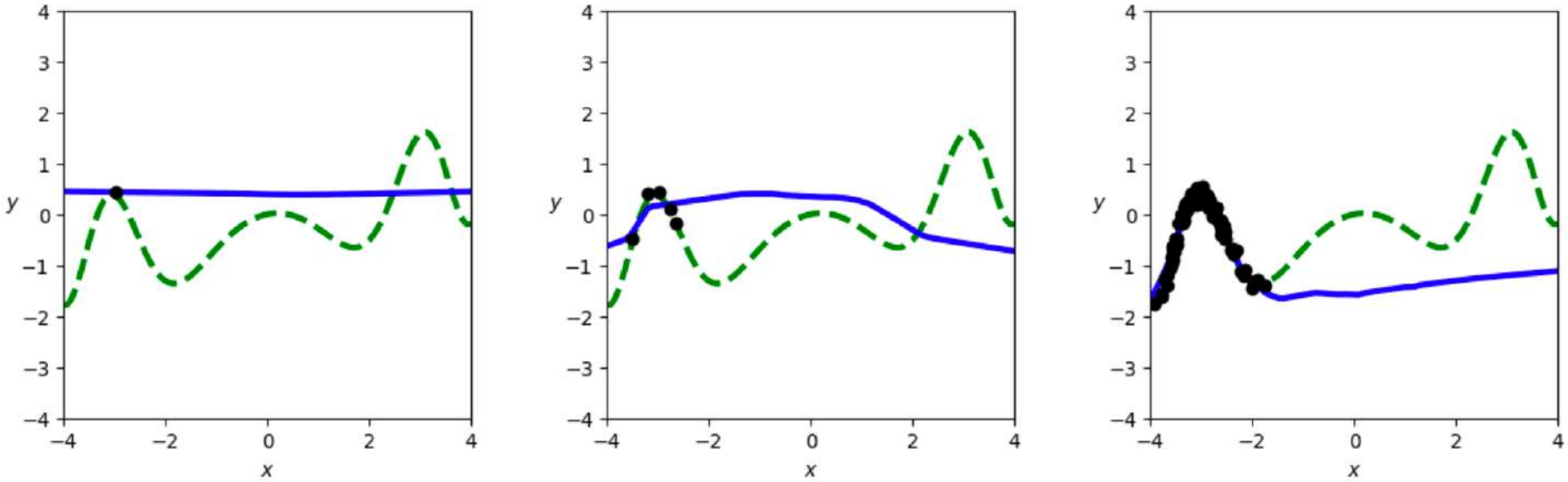
✖ Learned solution

Less data, better architecture.

We can pin down truth *either* by adding more data, or by using a more constrained architecture.

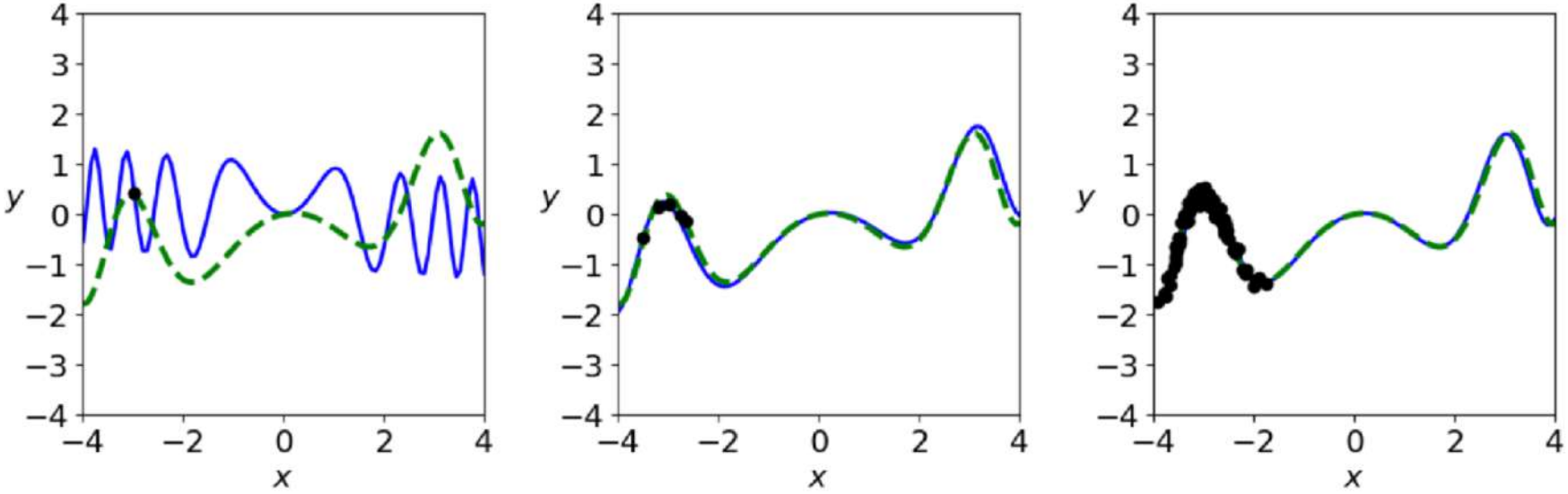
$$y = f(x)$$

f: 5 layer ReLU-net



- True solution
- Learned solution
- Training data

$$y = ax + \sin(bx^2)$$



- True solution
- Learned solution
- Training data

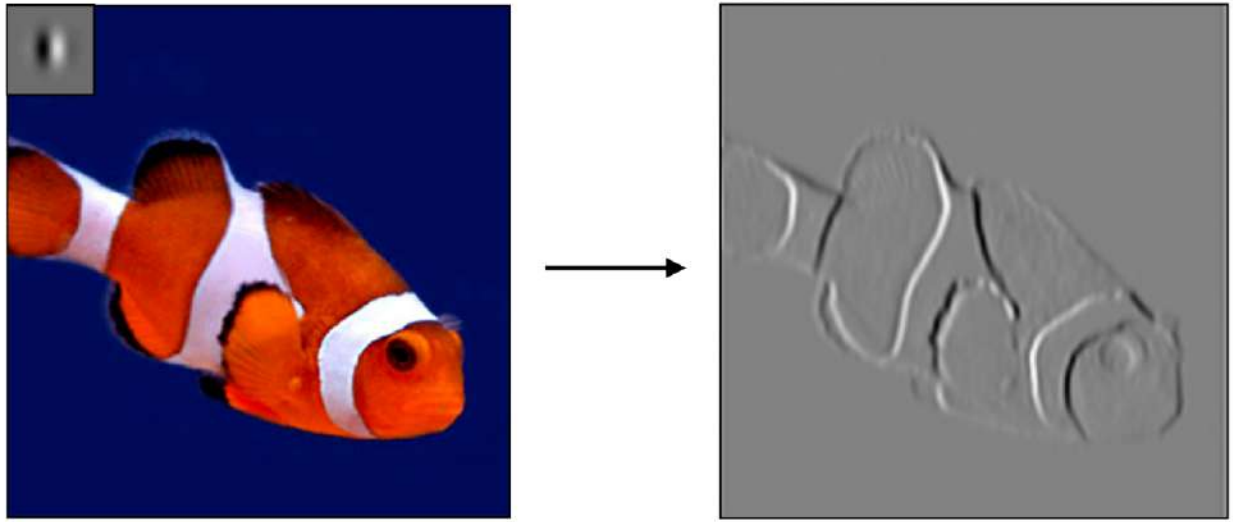
Architectures enable us to generalize *outside the training distribution*.

A good architecture is one that can represent the true function and is otherwise minimal (and is also easy to search over via gradient-based learning, easy to parallelize, fast on GPU, etc).

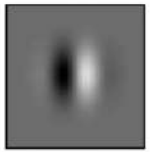
Convolutional Neural Networks

Convolution

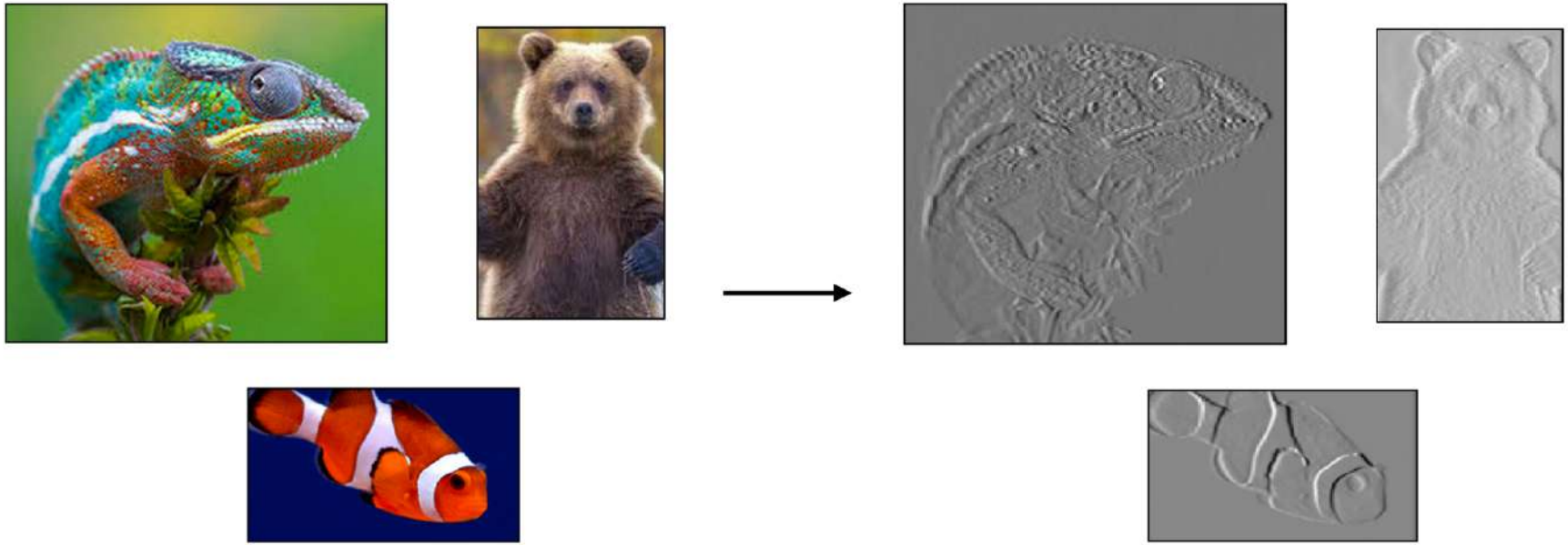
Linear, shift-invariant transformation



filter

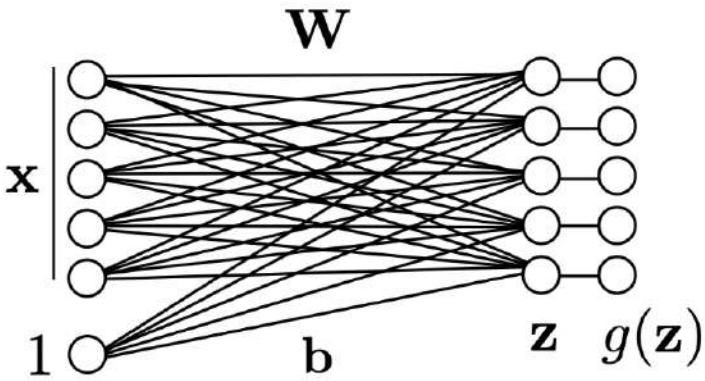


$$x_{\mathrm{out}}[n,m] = b + \sum_{k_1,k_2=-K}^{K} w[k_1,k_2]\, x_{\mathrm{in}}[n+k_1, m+k_2]$$

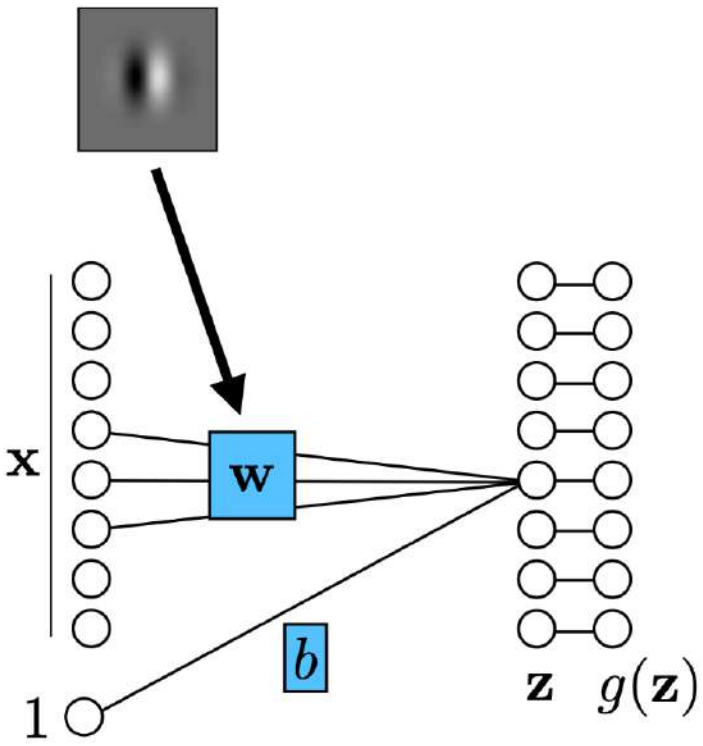


Fully-connected network

Fully-connected (fc) layer



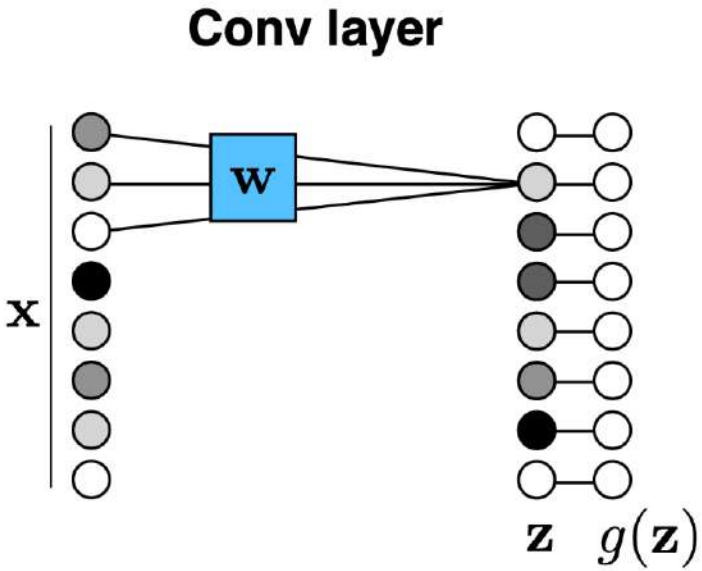
Convolutional neural network



Often, we assume output is a **local** function of input.

If we use the same weights (**weight sharing**) to compute each local function, we get a convolutional neural network.

Convolutional neural network

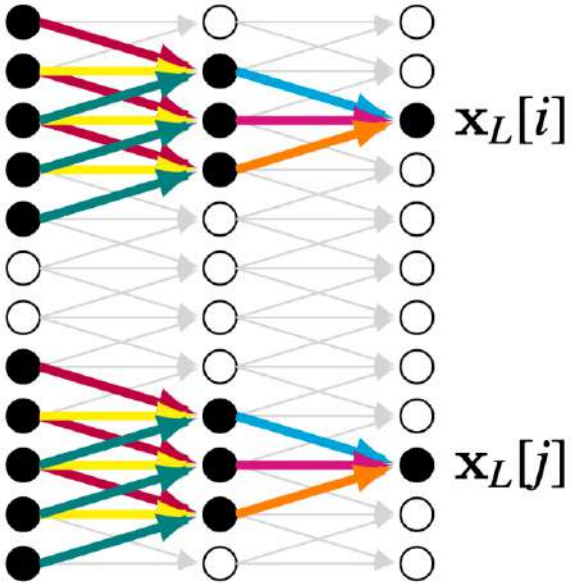


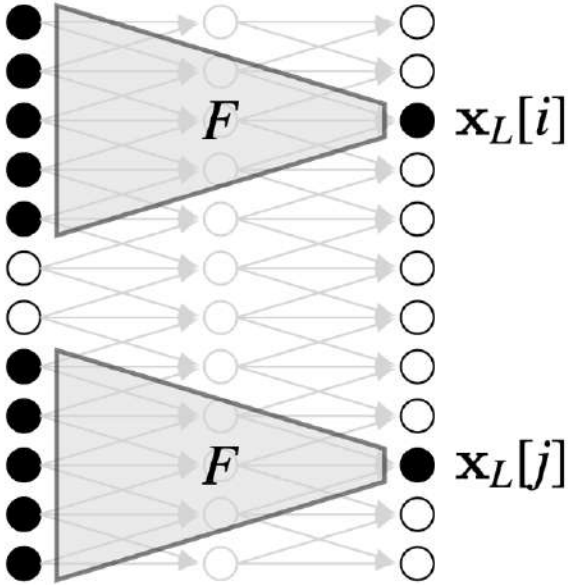
$$\boxed{\mathbf{z} = \mathbf{w} \star \mathbf{x} + b}$$

Often, we assume output is a **local** function of input.

If we use the same weights (**weight sharing**) to compute each local function, we get a convolutional neural network.

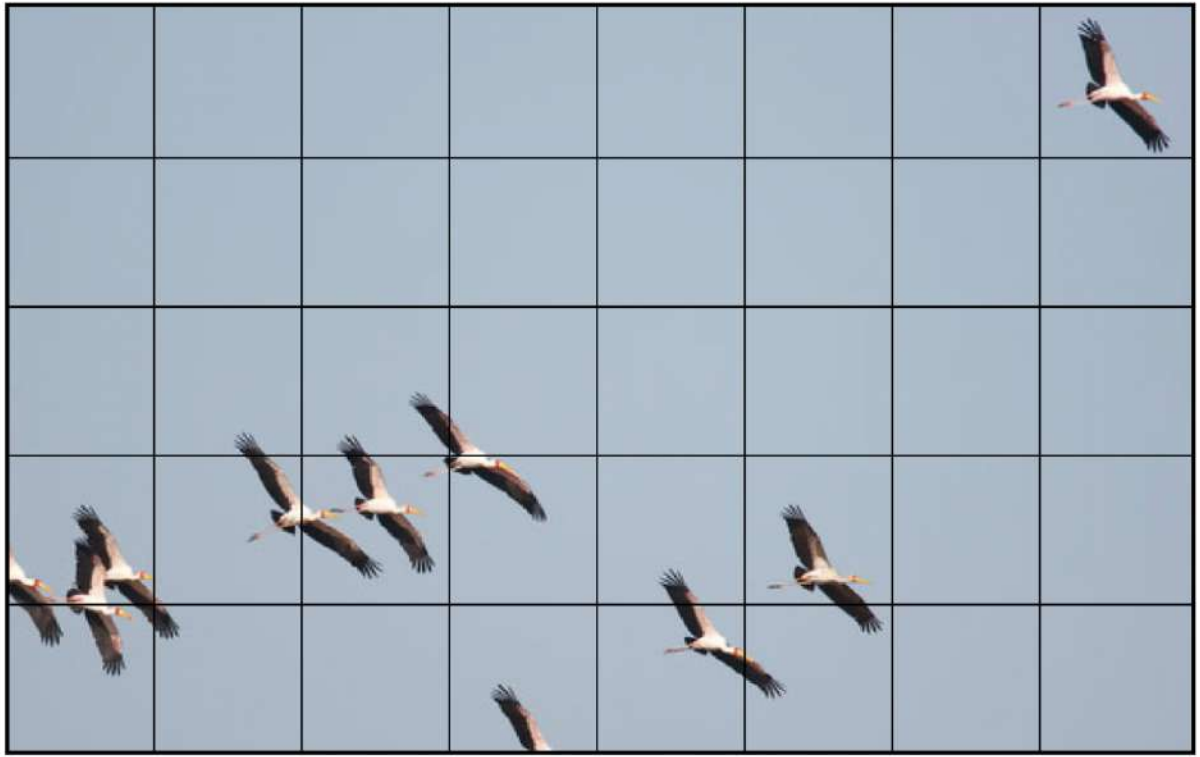
What happens when you stack convolutional layers?



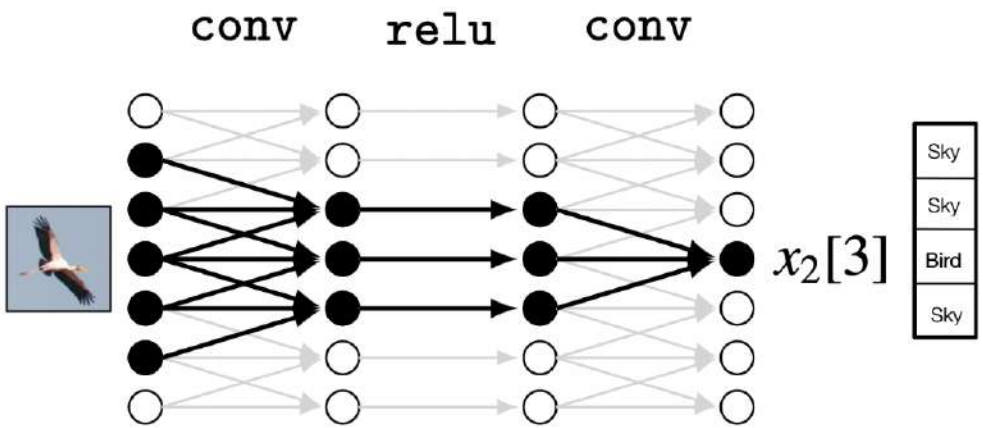


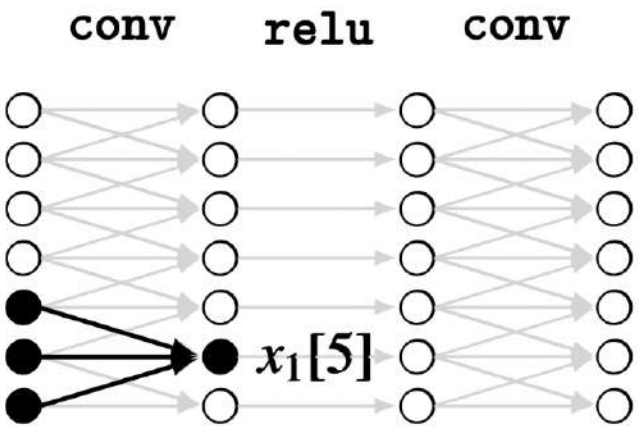
The whole CNN acts like a (nonlinear) convolutional filter!

Receptive fields



Receptive fields





Transformers

3 Key Ideas

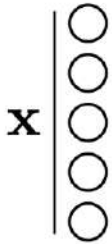
Idea #1: tokens

A new data structure: Tokens

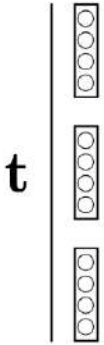
- A **token** is just transformer lingo for a vector of neurons (note: GNNs also operate over tokens, but over there we called them "node attributes" or node "feature descriptors")

- But the connotation is that a token is an encapsulated bundle of information; with transformers we will operate over tokens rather than over neurons

array of **neurons**



array of **tokens**

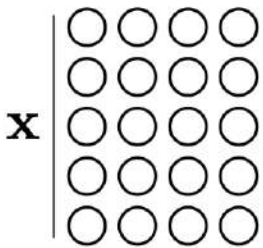


A new data structure: Tokens

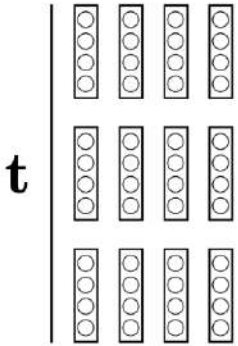
- A **token** is just transformer lingo for a vector of neurons (note: GNNs also operate over tokens, but over there we called them "node attributes" or node "feature descriptors")

- But the connotation is that a token is an encapsulated bundle of information; with transformers we will operate over tokens rather than over neurons

array of **neurons**



array of **tokens**

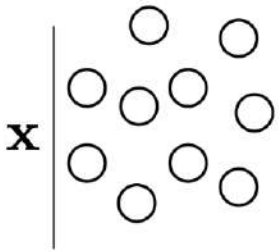


A new data structure: Tokens

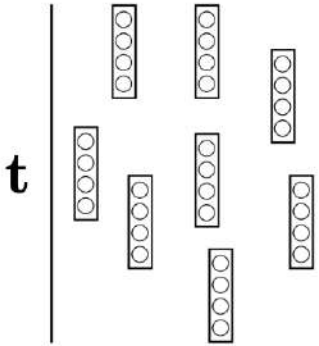
- A **token** is just transformer lingo for a vector of neurons (note: GNNs also operate over tokens, but over there we called them "node attributes" or node "feature descriptors")

- But the connotation is that a token is an encapsulated bundle of information; with transformers we will operate over tokens rather than over neurons

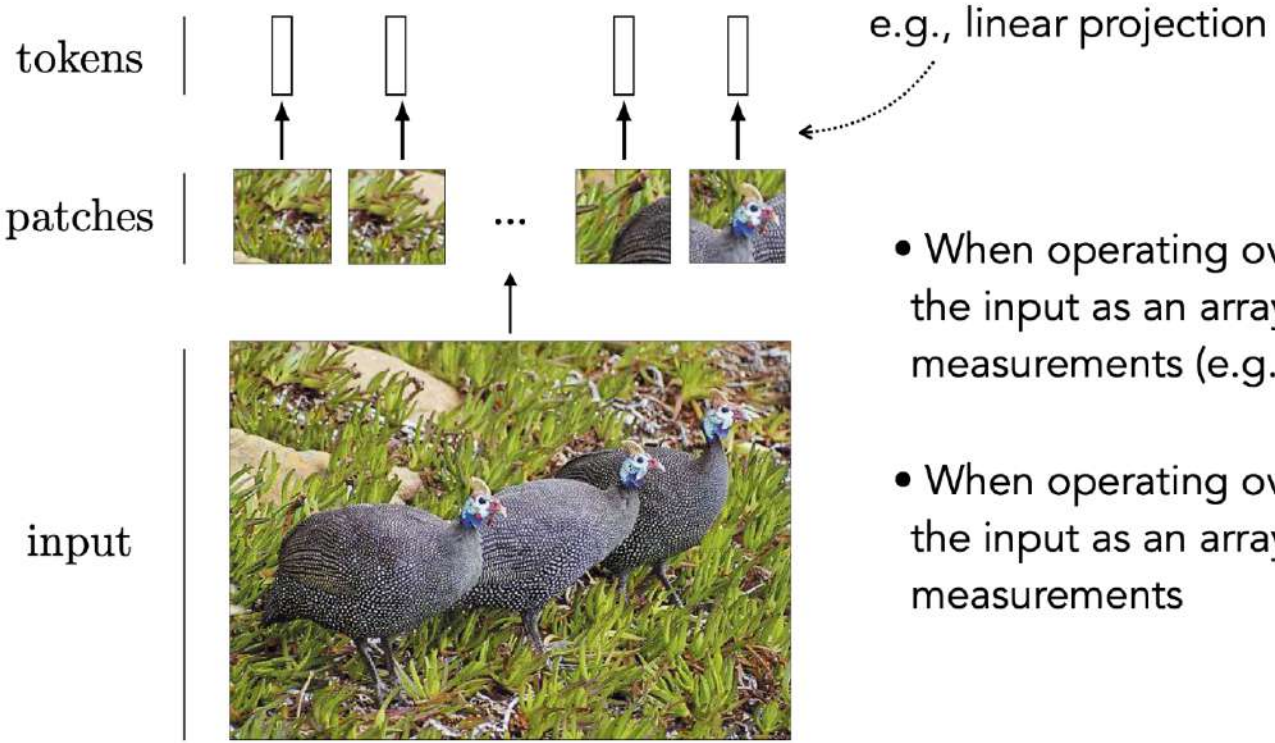
set of **neurons**



set of **tokens**



Tokenizing the input data



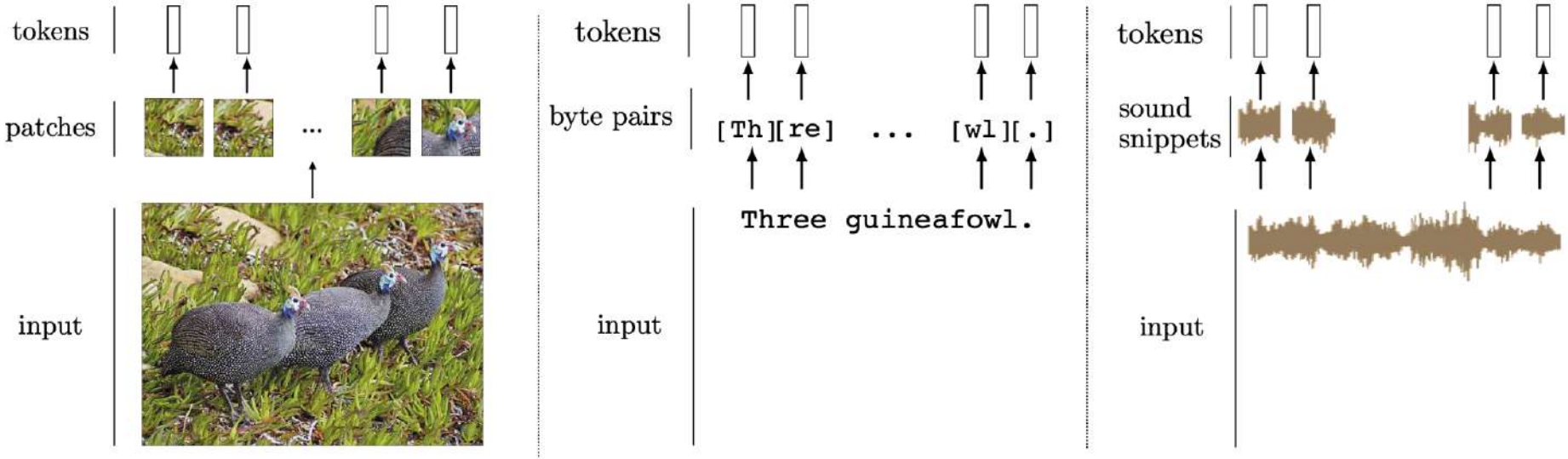
- When operating over *neurons*, we represent the input as an array of scalar-valued measurements (e.g., pixels)

- When operating over *tokens*, we represent the input as an array of vector-valued measurements

Tokenizing the input data

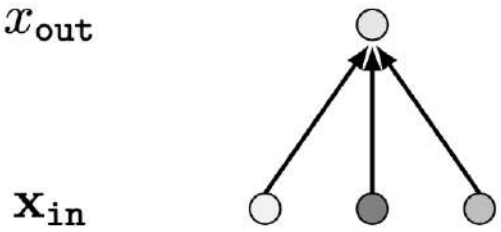
You can tokenize anything.

General strategy: chop the input up into chunks, project each chunk to a vector.



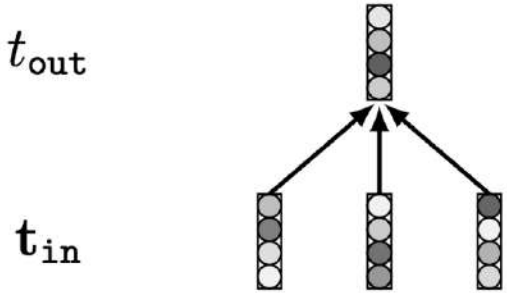
Linear combination of tokens

Linear combination of neurons



$$x_{\texttt{out}} = \sum_{i=1}^{N} w_i x_{\texttt{in}_i}$$

Linear combination of tokens



$$t_{\texttt{out}}.\mathbf{z} = \sum_{i=1}^{N} w_i t_{\texttt{in}_i}.\mathbf{z}$$

Token-wise nonlinearity

$$\mathbf{x}_{\text{out}} = [\texttt{relu}(x_1), \ldots, \texttt{relu}(x_N)]$$

$$\mathbf{t}_{\text{out}} = [F_\theta(t_1.\mathbf{z}), \ldots, F_\theta(t_N.\mathbf{z})]$$

F is typically an MLP

Equivalent to a CNN with 1x1 kernels run over token sequence

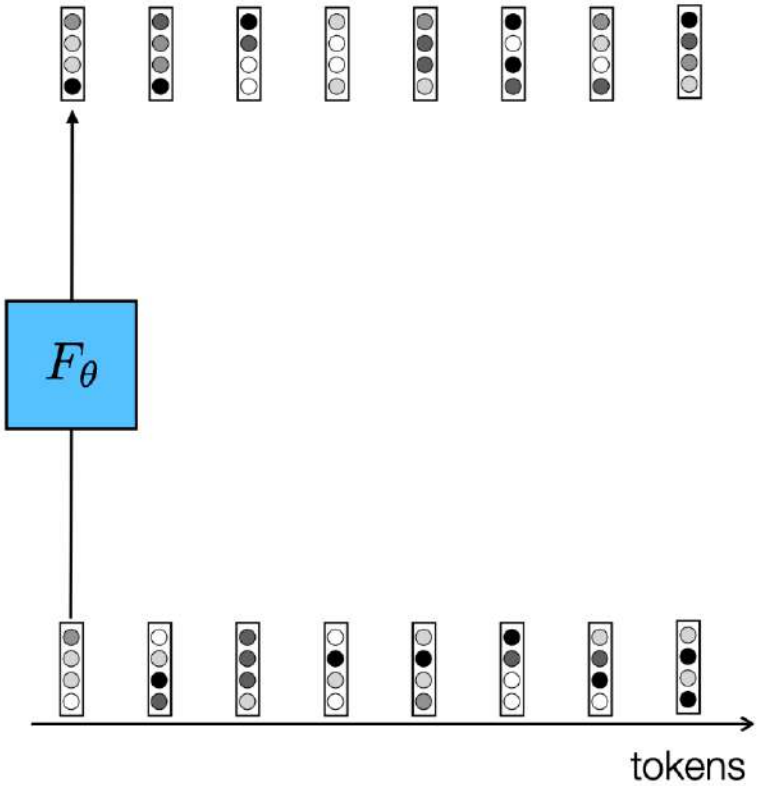
Token-wise nonlinearity

$$\mathbf{x}_{\text{out}} = [\texttt{relu}(x_1), \ldots, \texttt{relu}(x_N)]$$

$$\mathbf{t}_{\text{out}} = [F_\theta(t_1.\mathbf{z}), \ldots, F_\theta(t_N.\mathbf{z})]$$

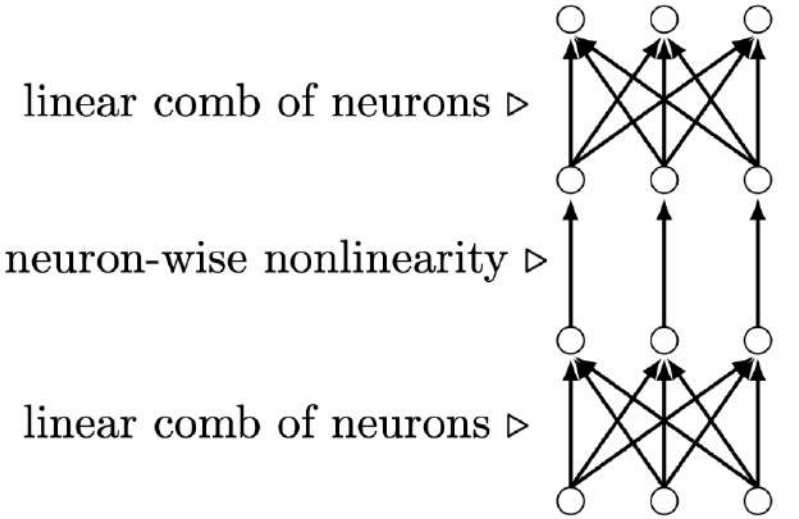
F is typically an MLP

Equivalent to a CNN with 1x1 kernels run over token sequence

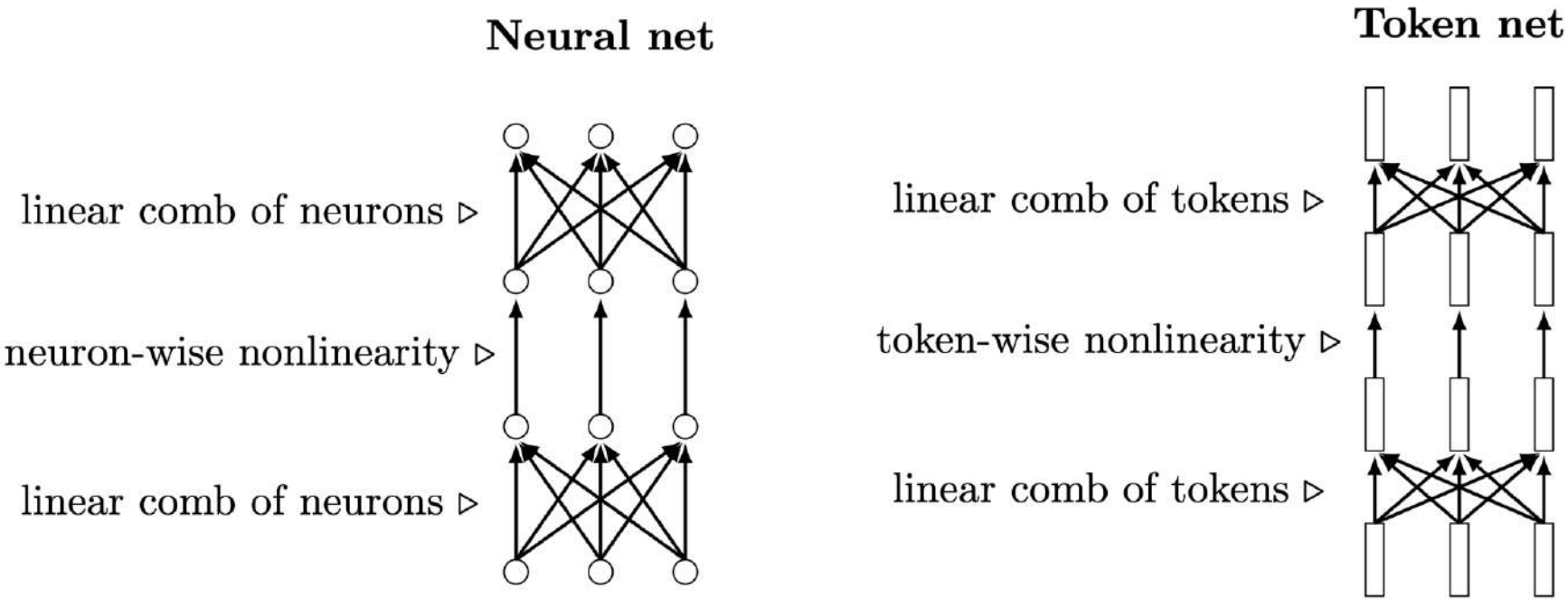


Token nets

Neural net



Token nets


Neural net
linear comb of neurons ▷
neuron-wise nonlinearity ▷
linear comb of neurons ▷
Token net
linear comb of tokens ▷
token-wise nonlinearity ▷
linear comb of tokens ▷

Idea #2: attention

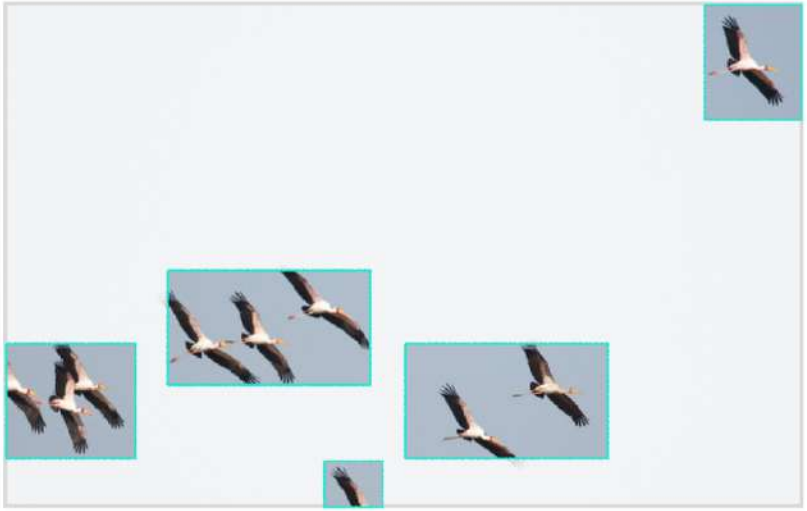
A limitation of CNNs

How many birds are in this image?

Is the top right bird the same species as the bottom left bird?

CNNs are built around the idea of locality, and are not well-suited to modeling long distance relationships

What is attention?

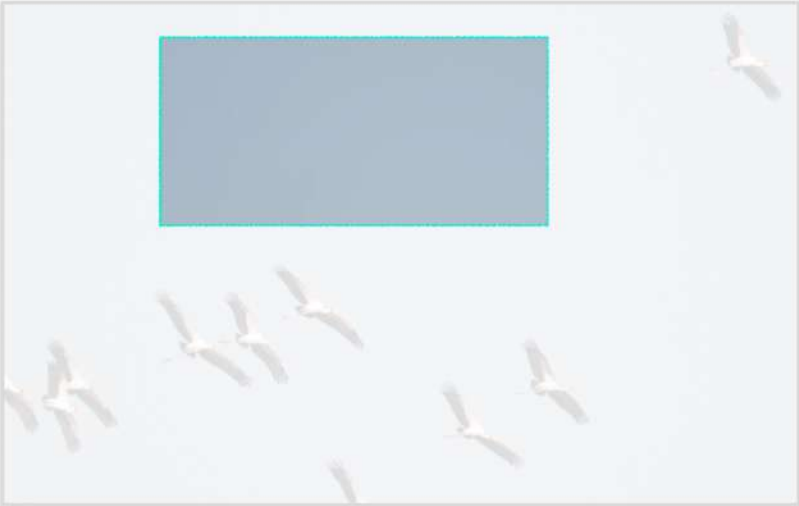


How many birds are in this image?

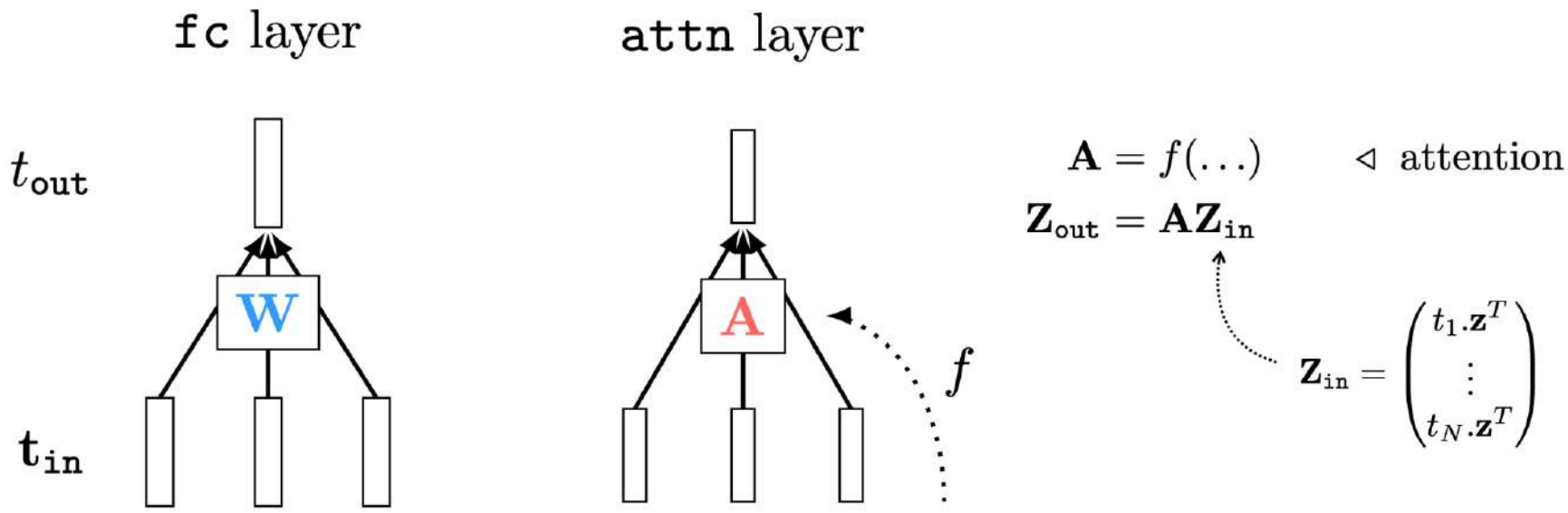
What is attention?

Is the top right bird the same species as the bottom left bird?

What is attention?



What's the color of the sky?



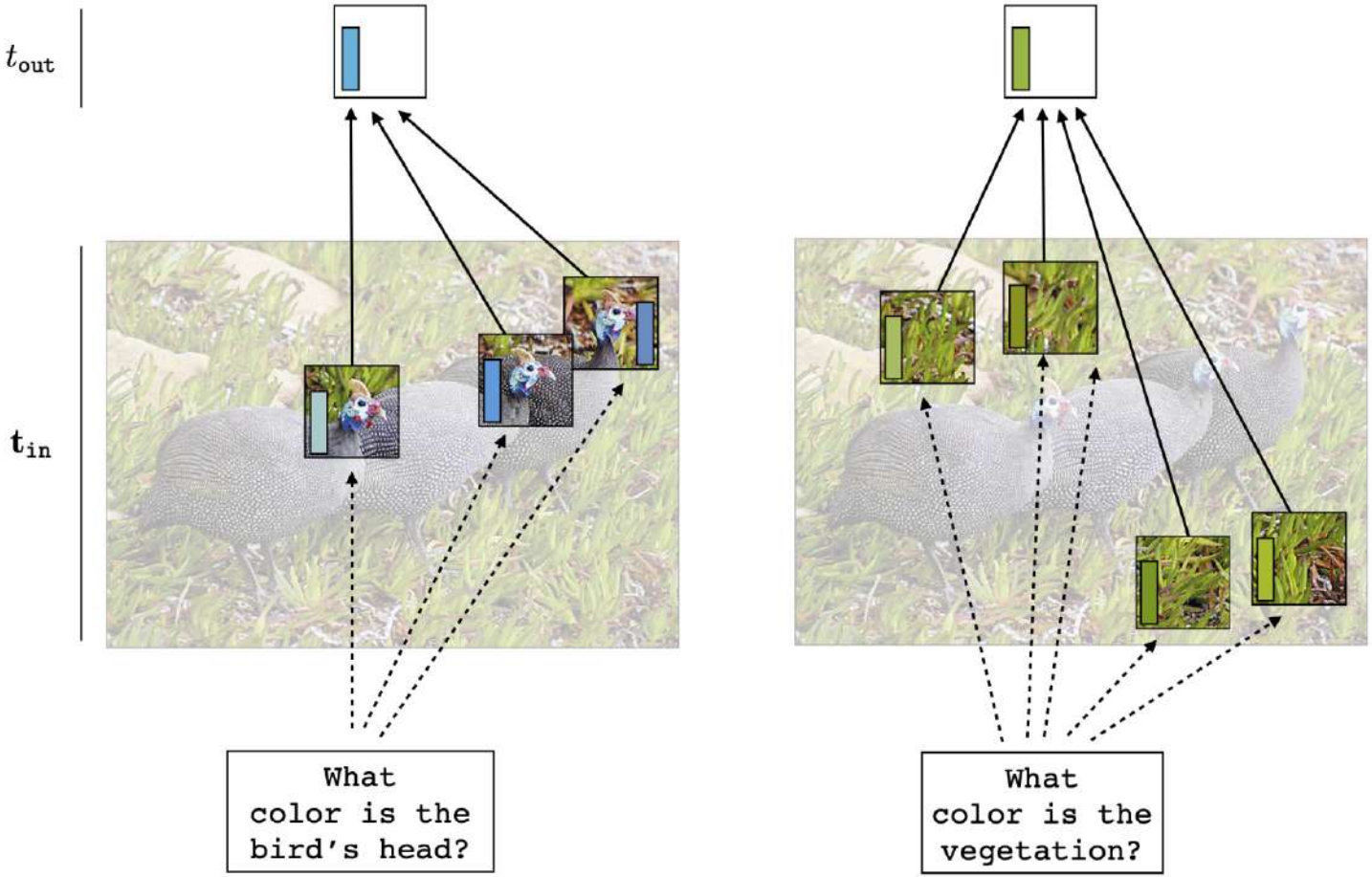
W is free *parameters*.

A is a function of some input *data*. The data tells us which tokens to attend to (assign high weight in weighted sum)

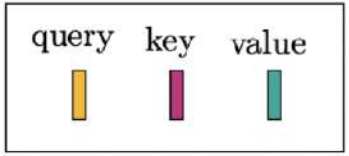
t_{out}

$\mathbf{t}_{\text{in}}$

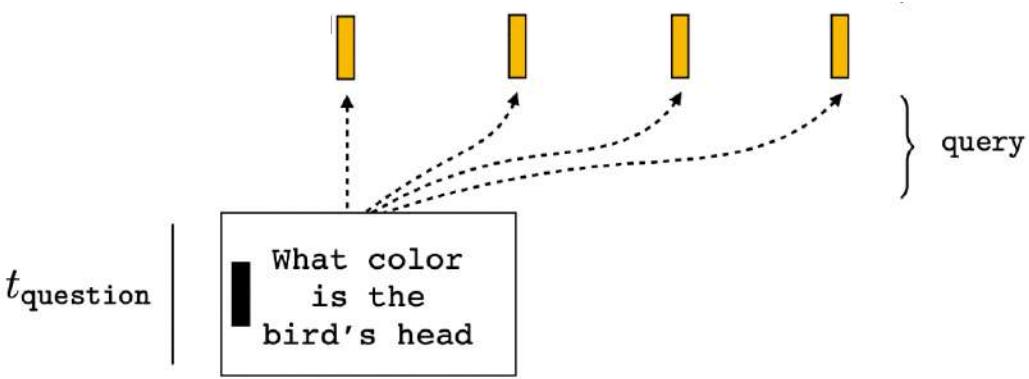
What color is the bird's head?


t_{out}
$\mathbf{t}_{\text{in}}$
What color is the bird's head?
What color is the vegetation?

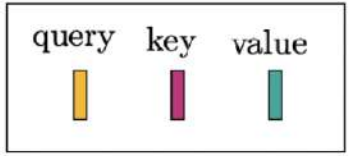
query-key-value attention



$$\mathbf{q} = t.\texttt{query}() = \mathbf{W}_q\mathbf{z}$$

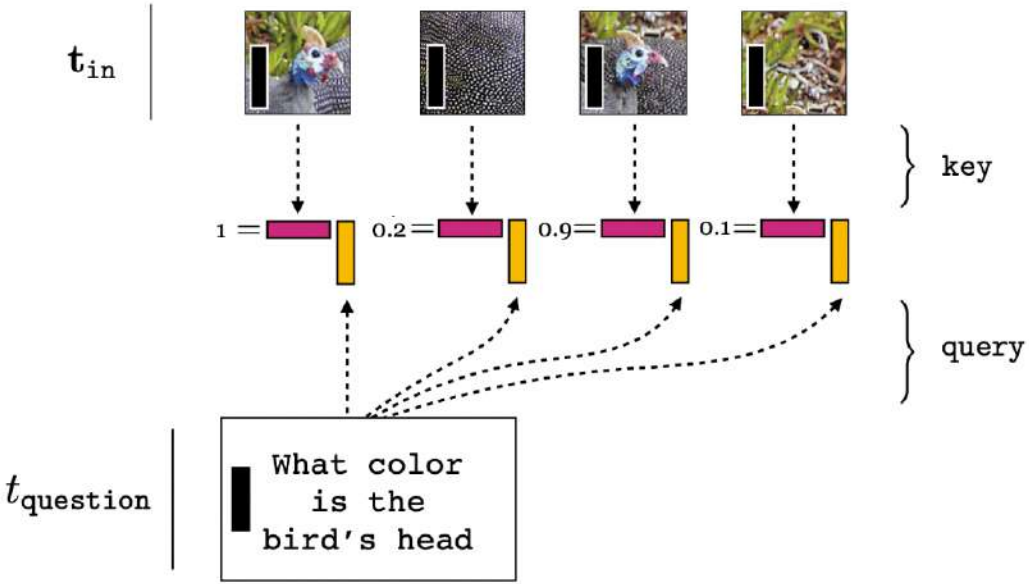


query-key-value attention

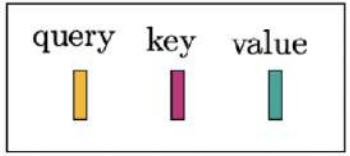


$$\mathbf{q} = t.\texttt{query}() = \mathbf{W}_q\mathbf{z}$$

$$\mathbf{k} = t.\texttt{key}() = \mathbf{W}_k\mathbf{z}$$



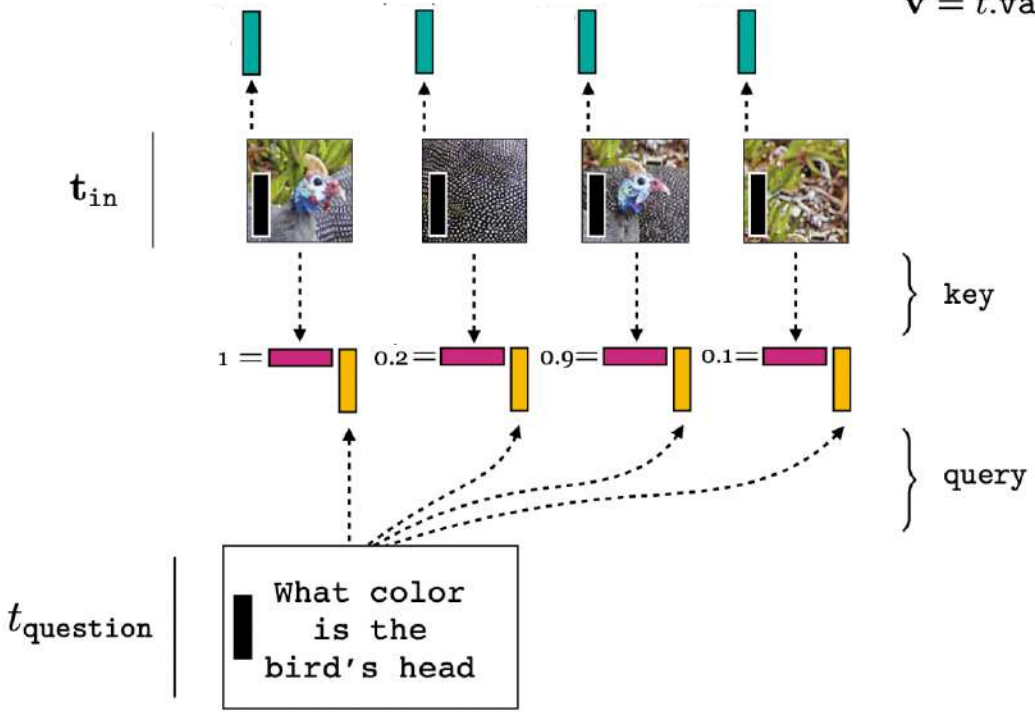
query-key-value attention



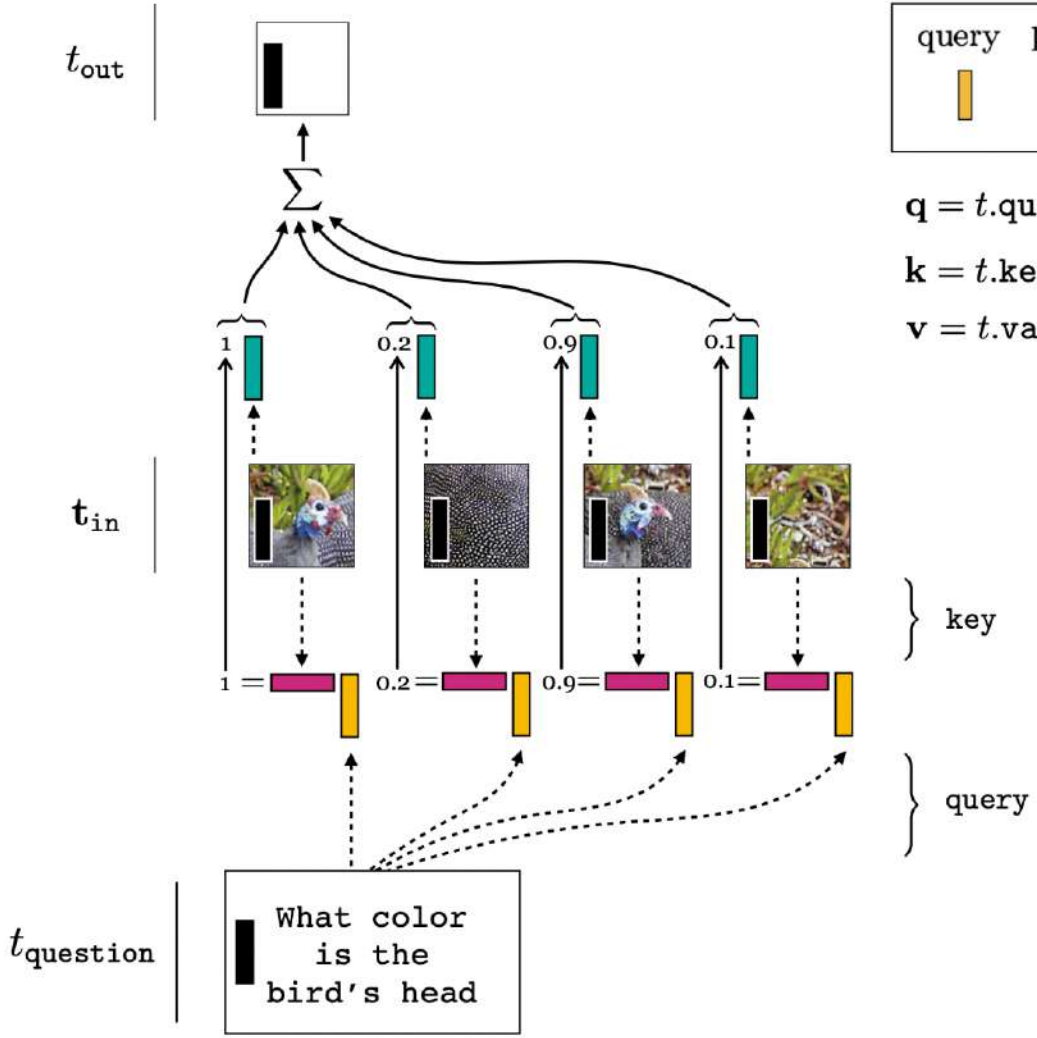
$$\mathbf{q} = t.\texttt{query}() = \mathbf{W}_q\mathbf{z}$$

$$\mathbf{k} = t.\texttt{key}() = \mathbf{W}_k\mathbf{z}$$

$$\mathbf{v} = t.\texttt{value}() = \mathbf{W}_v\mathbf{z}$$



query-key-value attention

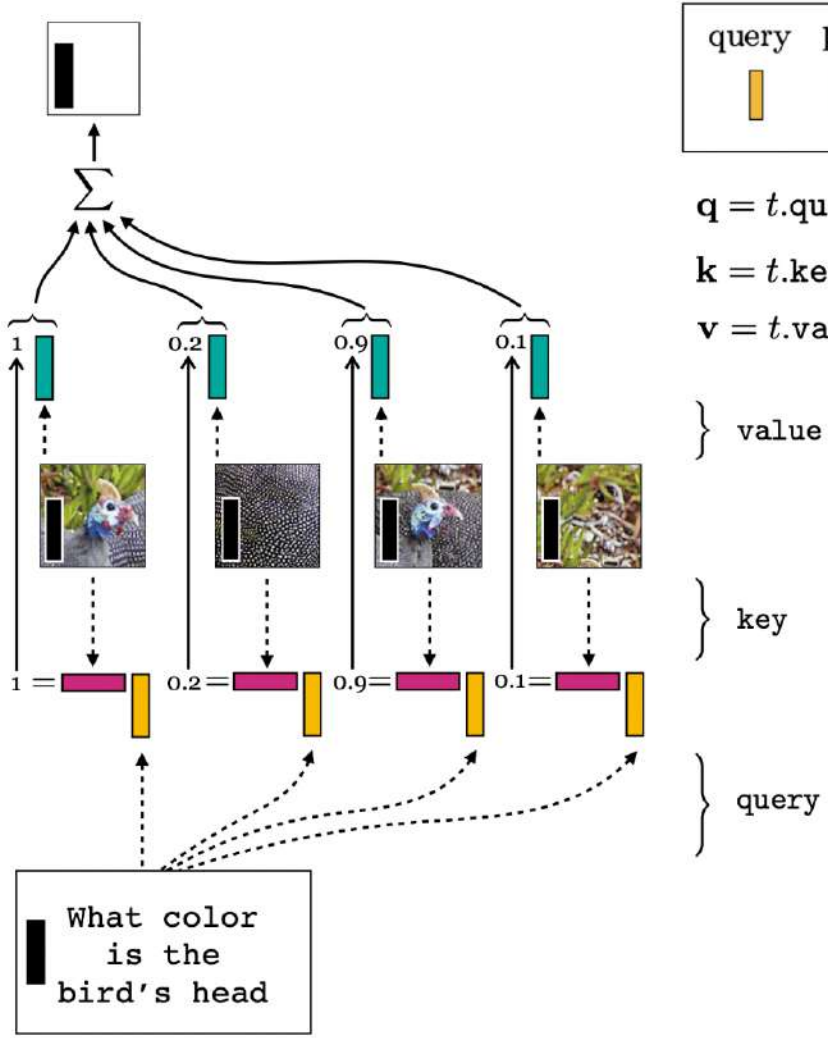


query-key-value attention

$$\mathbf{A} = \texttt{softmax}(\mathbf{s})$$
$$\mathbf{Z}_{\texttt{out}} = \mathbf{A}\mathbf{V}_{\texttt{in}}$$

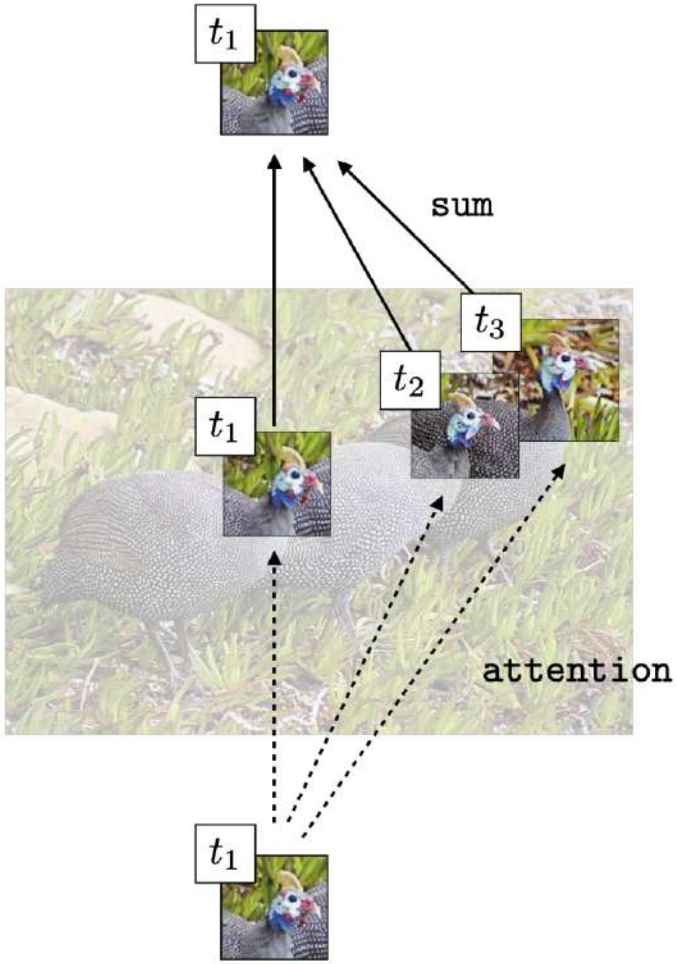
$$\mathbf{Q}_{\texttt{in}} = \begin{pmatrix} \mathbf{q}_1^T \\ \vdots \\ \mathbf{q}_N^T \end{pmatrix} \quad \mathbf{K}_{\texttt{in}} = \begin{pmatrix} \mathbf{k}_1^T \\ \vdots \\ \mathbf{k}_N^T \end{pmatrix} \quad \mathbf{V}_{\texttt{in}} = \begin{pmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_N^T \end{pmatrix}$$

$$\mathbf{s} = [\mathbf{q}_{\texttt{question}}^T \mathbf{k}_1, \ldots, \mathbf{q}_{\texttt{question}}^T \mathbf{k}_N]$$



$$\mathbf{q} = t.\texttt{query}() = \mathbf{W}_q\mathbf{z}$$
$$\mathbf{k} = t.\texttt{key}() = \mathbf{W}_k\mathbf{z}$$
$$\mathbf{v} = t.\texttt{value}() = \mathbf{W}_v\mathbf{z}$$

Self-attention



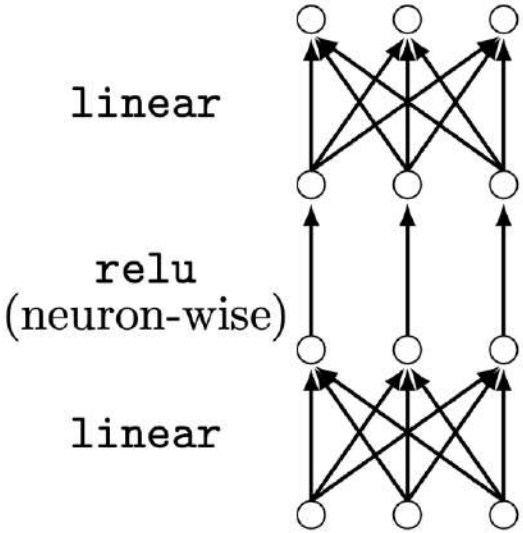
Attention maps in a trained transformer

[“DINO”, Caron et all. 2021]

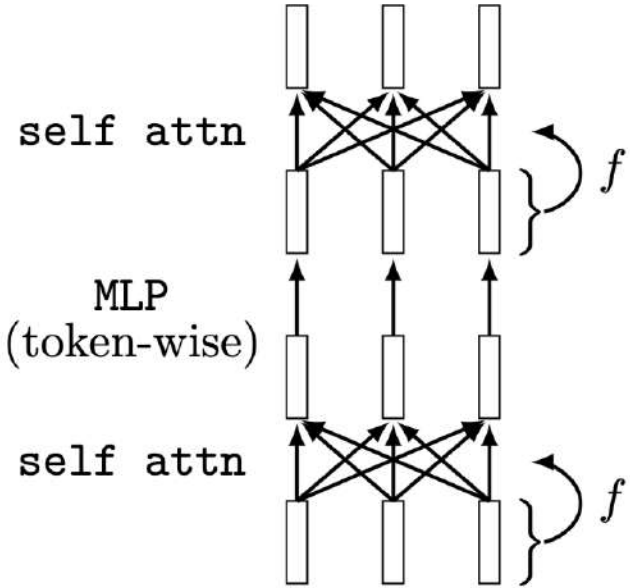
Attention maps in a trained transformer

[“DINO”, Caron et all. 2021]

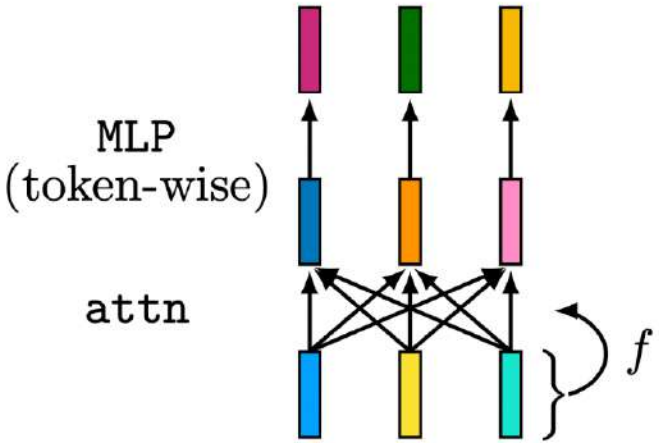
MLP

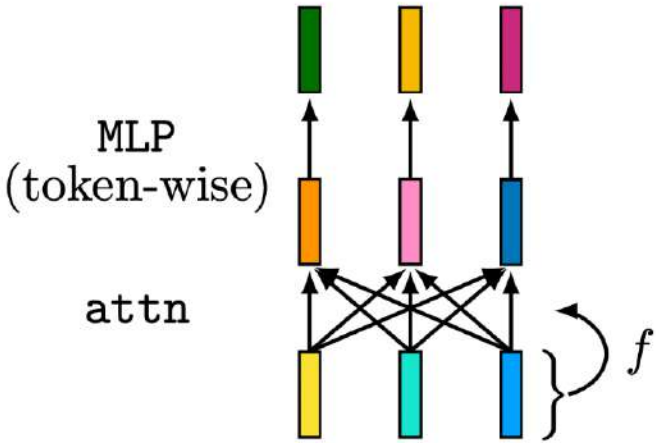


Transformer (vanilla)



Permutation equivariance





$$\texttt{attn}(\texttt{permute}(\mathbf{t}_{\texttt{in}})) = \texttt{permute}(\texttt{attn}(\mathbf{t}_{\texttt{in}}))$$

$$\texttt{tokenMLP}(\texttt{permute}(\mathbf{t}_{\texttt{in}})) = \texttt{tokenMLP}(\texttt{attn}(\mathbf{t}_{\texttt{in}}))$$

$$\downarrow$$

$$\texttt{transformer}(\texttt{permute}(\mathbf{t}_{\texttt{in}})) = \texttt{permute}(\texttt{transformer}(\mathbf{t}_{\texttt{in}}))$$

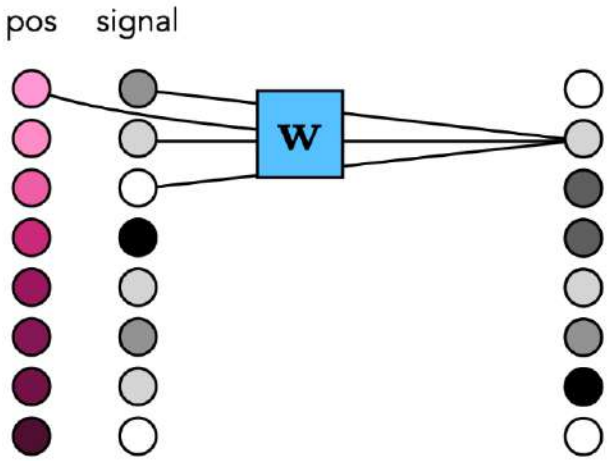
Set2Set

Idea #3: positional encoding

What if you *don't* want to be shift invariant?

1. Use an architecture that is not shift invariant (e.g., MLP)

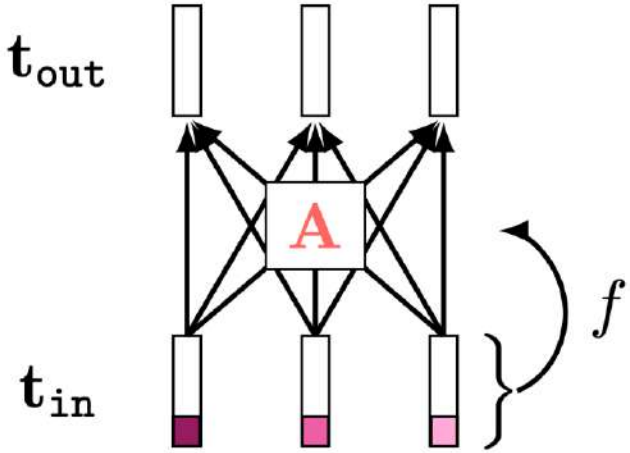
2. Add location information to the *input* to the convolutional filters — this is called **positional encoding**



What if you *don't* want to be permutation invariant?

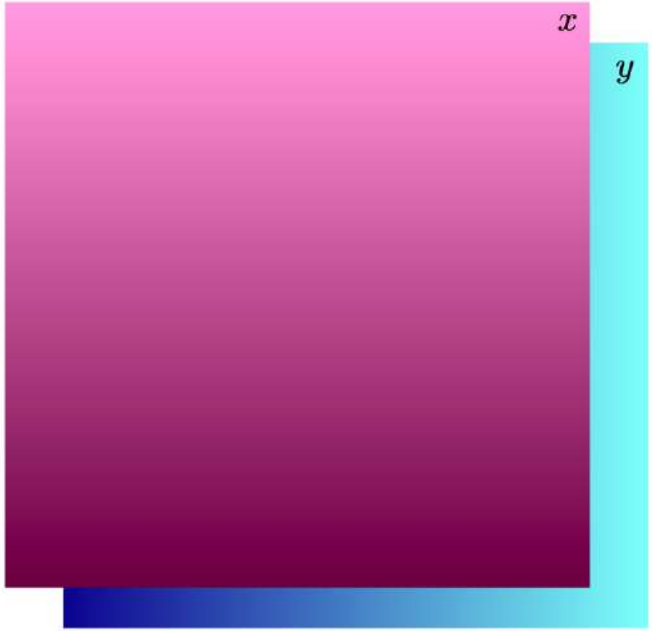
1. Use an architecture that is not permutation invariant (e.g., MLP)

2. Add location information to the token code vectors — this is called **positional encoding**



Neural Fields

Coordinates



$$\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}$$

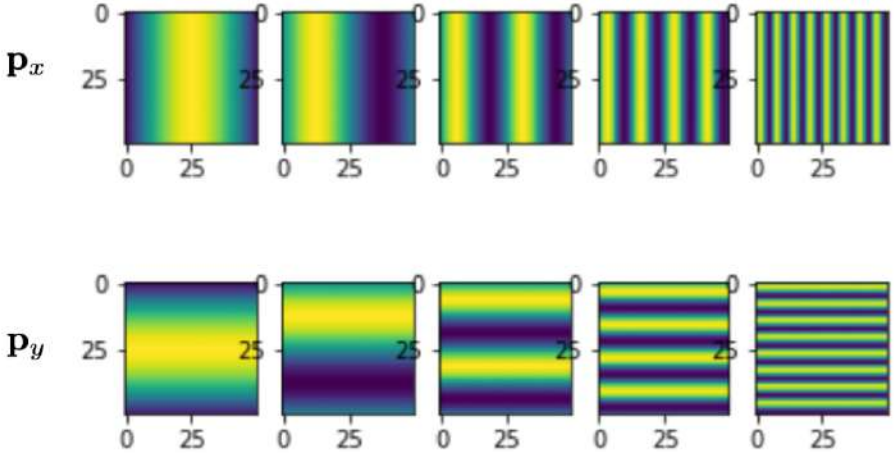
Field

$$l = \Phi(x, y)$$

Fourier positional codes

Represent coordinates on Fourier basis

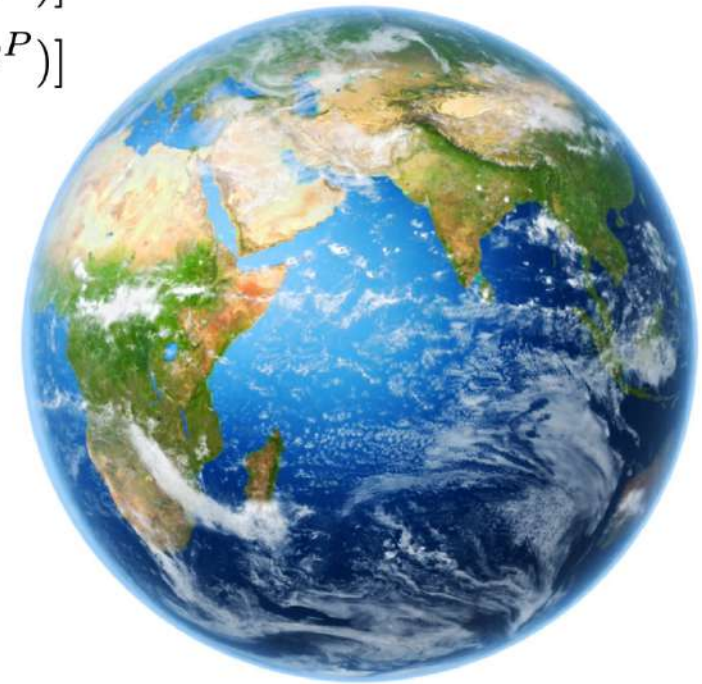
$$\mathbf{p}_x = [\sin(x), \sin(x/B), \sin(x/B^2), \ldots, \sin(x/B^P)]$$
$$\mathbf{p}_y = [\sin(y), \sin(y/B), \sin(y/B^2), \ldots, \sin(y/B^P)]$$



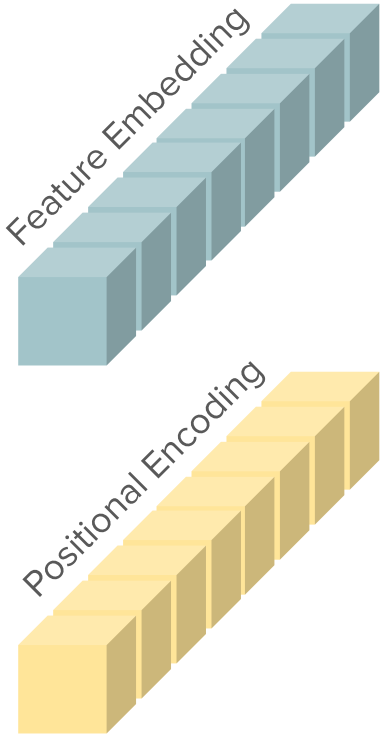
Geospatial Positional Encoding

$$\mathbf{p}_x = [\sin(x), \sin(x/B), \sin(x/B^2), \ldots, \sin(x/B^P)]$$

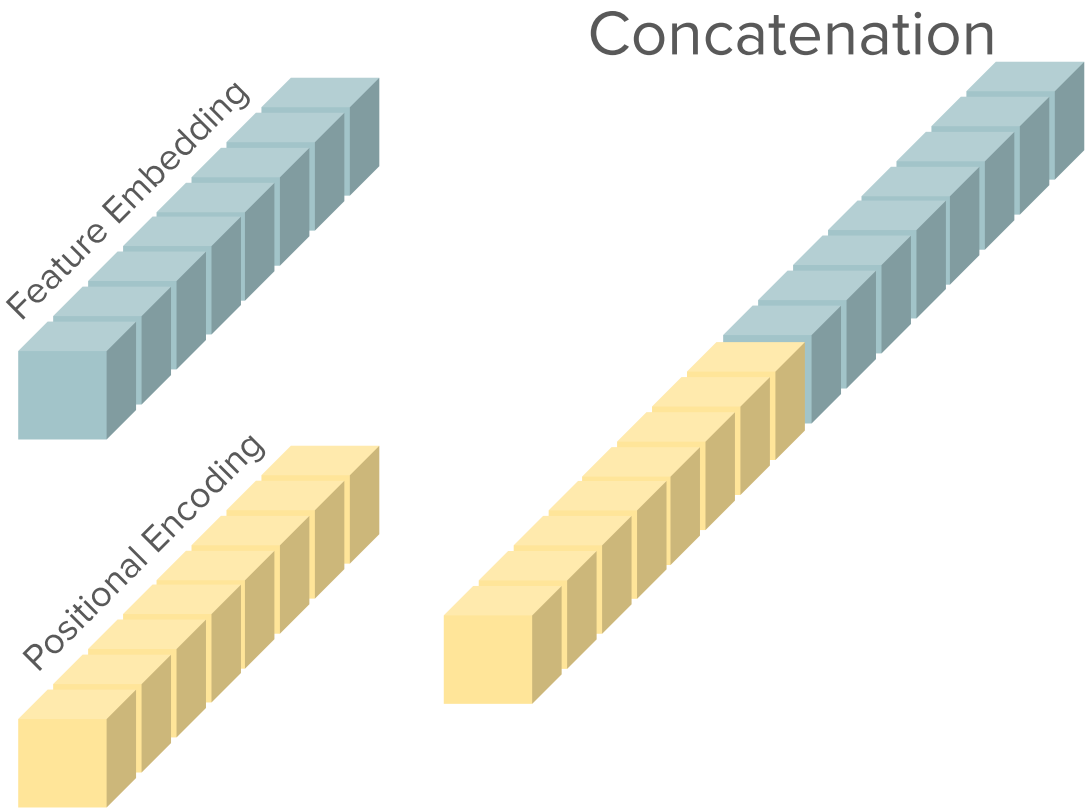
$$\mathbf{p}_y = [\sin(y), \sin(y/B), \sin(y/B^2), \ldots, \sin(y/B^P)]$$



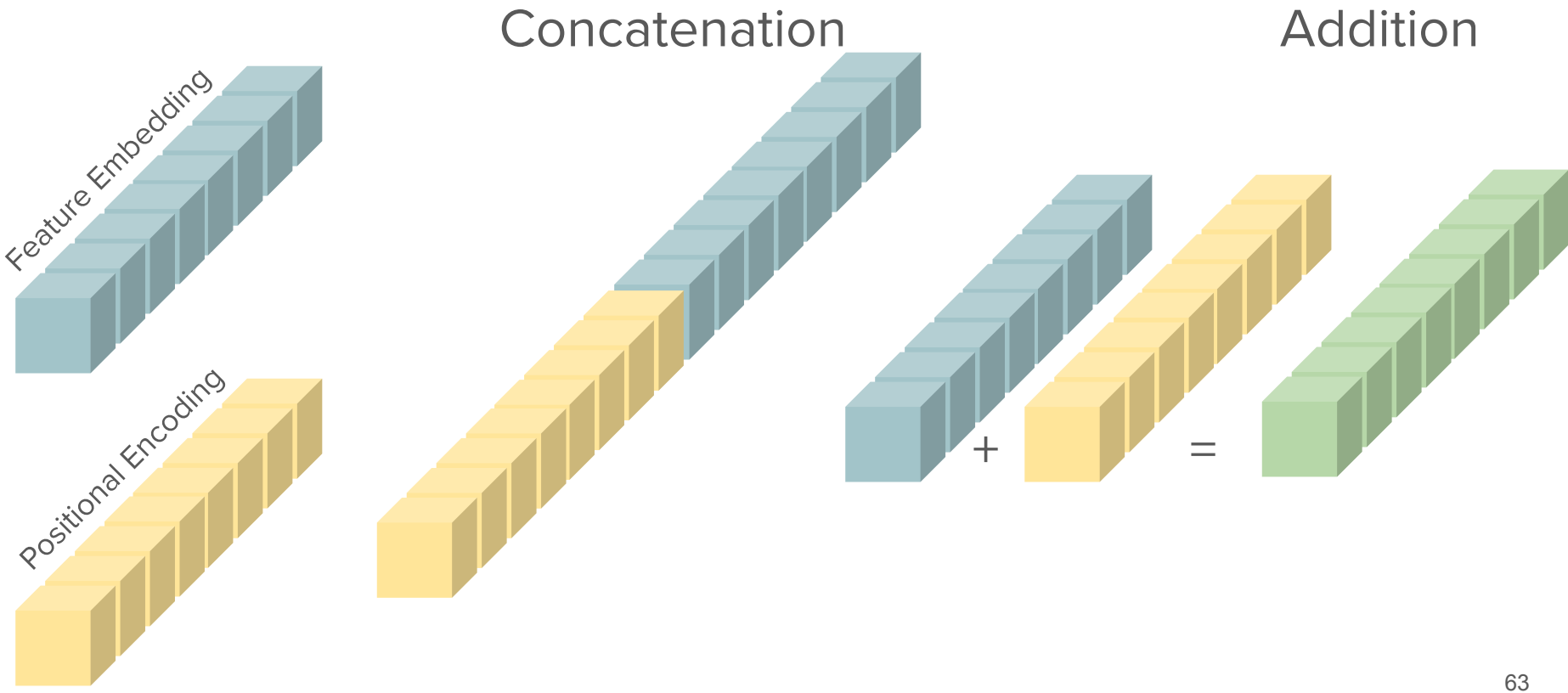
Applying positional embeddings



Applying positional embeddings



Applying positional embeddings



RoPE: Rotary Position Embedding

$$d = 16 \quad \begin{bmatrix} -0.53 \\ -1.35 \\ 0.05 \\ -0.98 \\ -0.33 \\ 0.03 \\ -2.22 \\ 0.42 \\ -0.46 \\ -0.10 \\ -1.56 \\ 0.42 \\ 0.85 \\ -0.08 \\ -1.10 \\ -0.18 \end{bmatrix} \xrightarrow[\text{for 2D rotations}]{\text{Split into pairs}} \begin{bmatrix} \boxed{\begin{matrix} -0.53 \\ -1.35 \end{matrix}} \\ \boxed{\begin{matrix} 0.05 \\ -0.98 \end{matrix}} \\ \boxed{\begin{matrix} -0.33 \\ 0.03 \end{matrix}} \\ \boxed{\begin{matrix} -2.22 \\ 0.42 \end{matrix}} \\ \boxed{\begin{matrix} -0.46 \\ -0.10 \end{matrix}} \\ \boxed{\begin{matrix} -1.56 \\ 0.42 \end{matrix}} \\ \boxed{\begin{matrix} 0.85 \\ -0.08 \end{matrix}} \\ \boxed{\begin{matrix} -1.10 \\ -0.18 \end{matrix}} \end{bmatrix} \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix} \quad K = 8$$

RoPE: Rotary Position Embedding

$d = 16$
$K = 8$
$\theta = 1000$

$$\omega_k = \frac{1}{\theta^{2k/d}} \longrightarrow \begin{bmatrix} 1.0 \\ 0.3162278 \\ 0.1 \\ 0.0316228 \\ 0.01 \\ 0.0031623 \\ 0.001 \\ 0.0003162 \end{bmatrix}$$

RoPE: Rotary Position Embedding

"This sentence has five words"

0	1	2	3	4
0.0	1.0	2.0	3.0	4.0
0.0	0.3162	0.6325	0.9487	1.2649
0.0	0.1	0.2	0.3	0.4
0.0	0.0316	0.0632	0.0949	0.1265
0.0	0.01	0.02	0.03	0.04
0.0	0.0032	0.0063	0.0095	0.0126
0.0	0.001	0.002	0.003	0.004
0.0	0.0003	0.0006	0.0009	0.0013

$\omega_k = \frac{1}{\theta^{2k/d}} \quad d = 16 \quad \theta = 10000$

RoPE: Rotary Position Embedding

$$2$$

$$\begin{bmatrix} 0.5403 & -0.8415 \\ 0.8415 & 0.5403 \end{bmatrix}$$

$$\begin{bmatrix} 1.0 \\ 0.3162278 \\ 0.1 \\ 0.0316228 \\ 0.01 \\ 0.0031623 \\ 0.001 \\ 0.0003162 \end{bmatrix} \quad \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix}$$

RoPE: Rotary Position Embedding

$$2$$

$$\begin{bmatrix} 0.5403 & -0.8415 \\ 0.8415 & 0.5403 \end{bmatrix}$$

$$\begin{bmatrix} 0.9504 & -0.3110 \\ 0.3110 & 0.9504 \end{bmatrix}$$

$$\begin{bmatrix} 1.0 \\ 0.3162278 \\ 0.1 \\ 0.0316228 \\ 0.01 \\ 0.0031623 \\ 0.001 \\ 0.0003162 \end{bmatrix}$$

$$\begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix}$$

RoPE: Rotary Position Embedding

$$\begin{bmatrix} 1.0 \\ 0.3162278 \\ 0.1 \\ 0.0316228 \\ 0.01 \\ 0.0031623 \\ 0.001 \\ 0.0003162 \end{bmatrix} \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix}$$

2

$$\begin{bmatrix} 0.5403 & -0.8415 \\ 0.8415 & 0.5403 \end{bmatrix}$$
$$\begin{bmatrix} 0.9504 & -0.3110 \\ 0.3110 & 0.9504 \end{bmatrix}$$
$$\begin{bmatrix} 0.9950 & -0.0998 \\ 0.0998 & 0.9950 \end{bmatrix}$$
$$\begin{bmatrix} 0.9995 & -0.0316 \\ 0.0316 & 0.9995 \end{bmatrix}$$
$$\begin{bmatrix} 1.0000 & -0.0100 \\ 0.0100 & 1.0000 \end{bmatrix}$$
$$\begin{bmatrix} 1.0000 & -0.0032 \\ 0.0032 & 1.0000 \end{bmatrix}$$
$$\begin{bmatrix} 1.0000 & -0.0010 \\ 0.0010 & 1.0000 \end{bmatrix}$$
$$\begin{bmatrix} 1.0000 & -0.0003 \\ 0.0003 & 1.0000 \end{bmatrix}$$

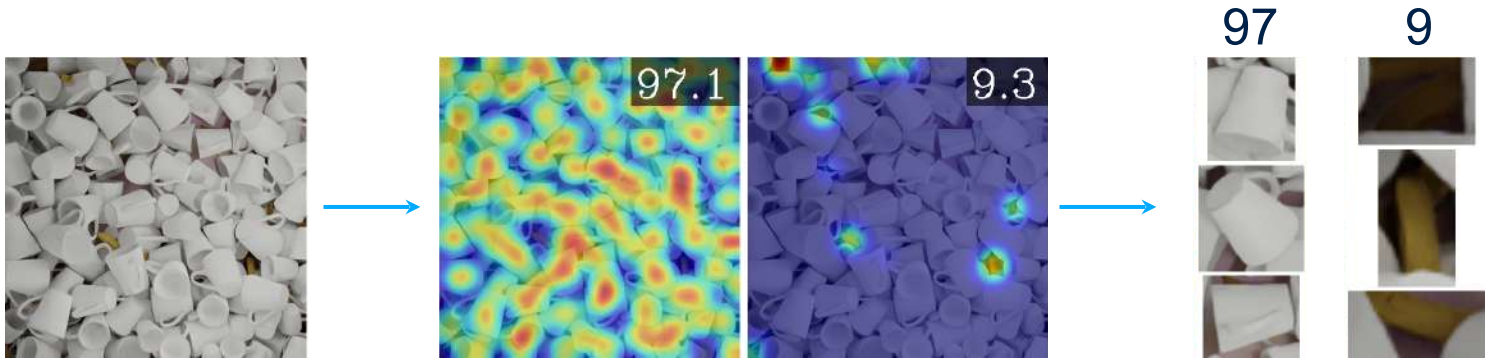
16

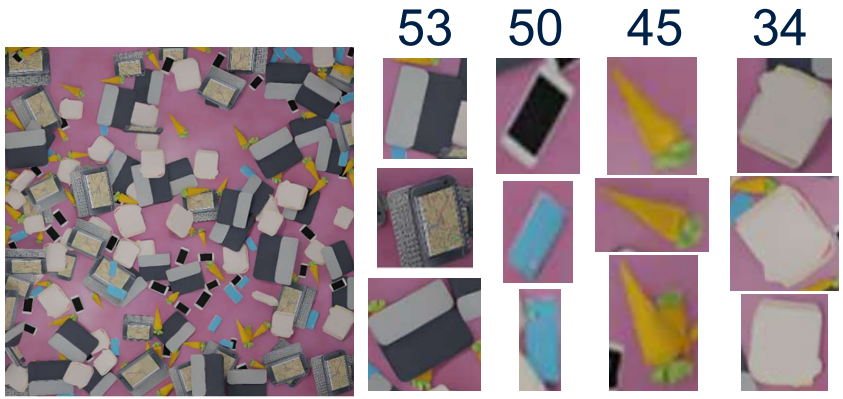
$$\begin{bmatrix} -0.53 \\ -1.35 \\ 0.05 \\ -0.98 \\ -0.33 \\ 0.03 \\ -2.22 \\ 0.42 \\ -0.46 \\ -0.10 \\ -1.56 \\ 0.42 \\ 0.85 \\ -0.08 \\ -1.10 \\ -0.18 \end{bmatrix}$$

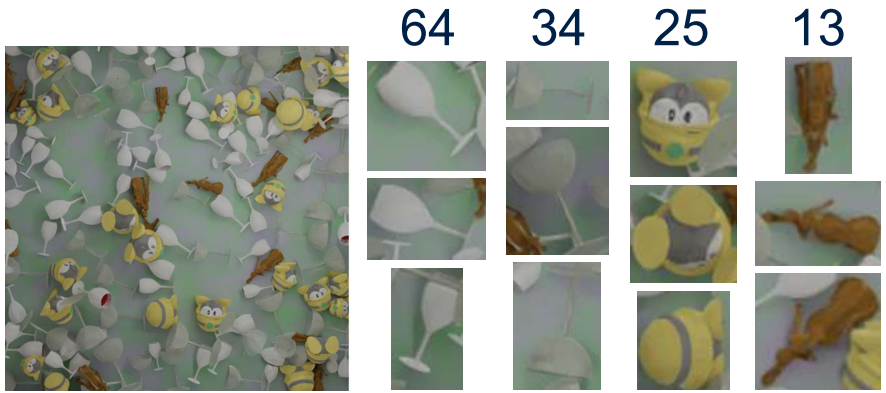
Same dimensionality as the original embedding

Recent Applications of ViTs Tasks and Encoders

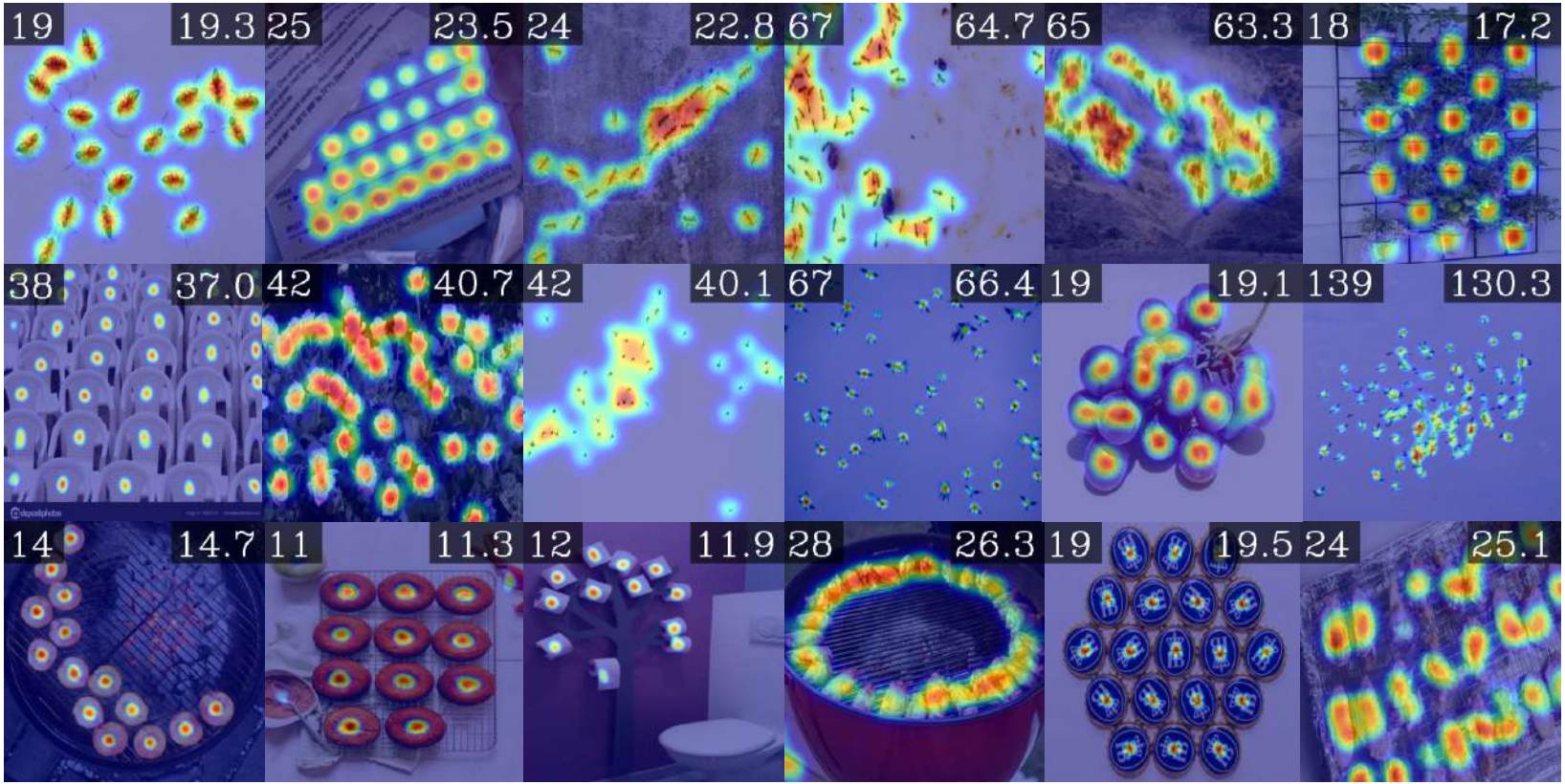
Counting: ABC123



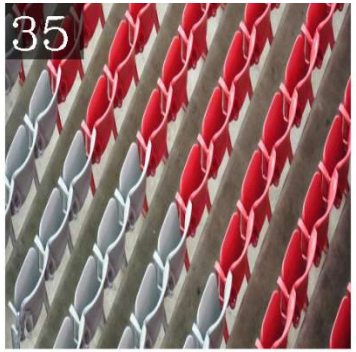




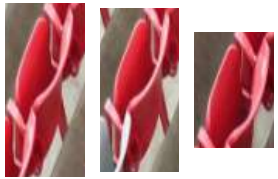
Counting: ABC123



Counting: ABC123



17



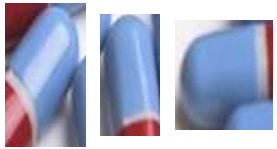
16

17

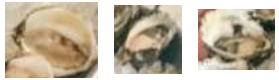
19

23

23



15



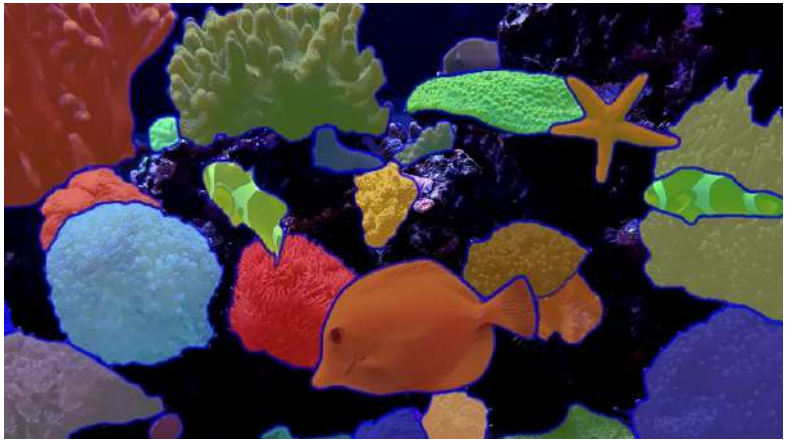
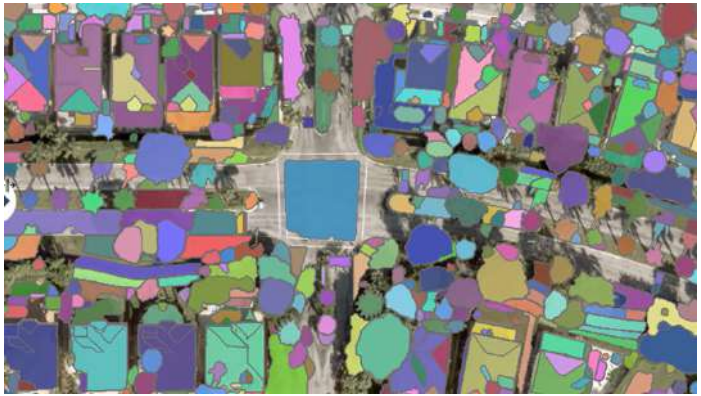
7

19

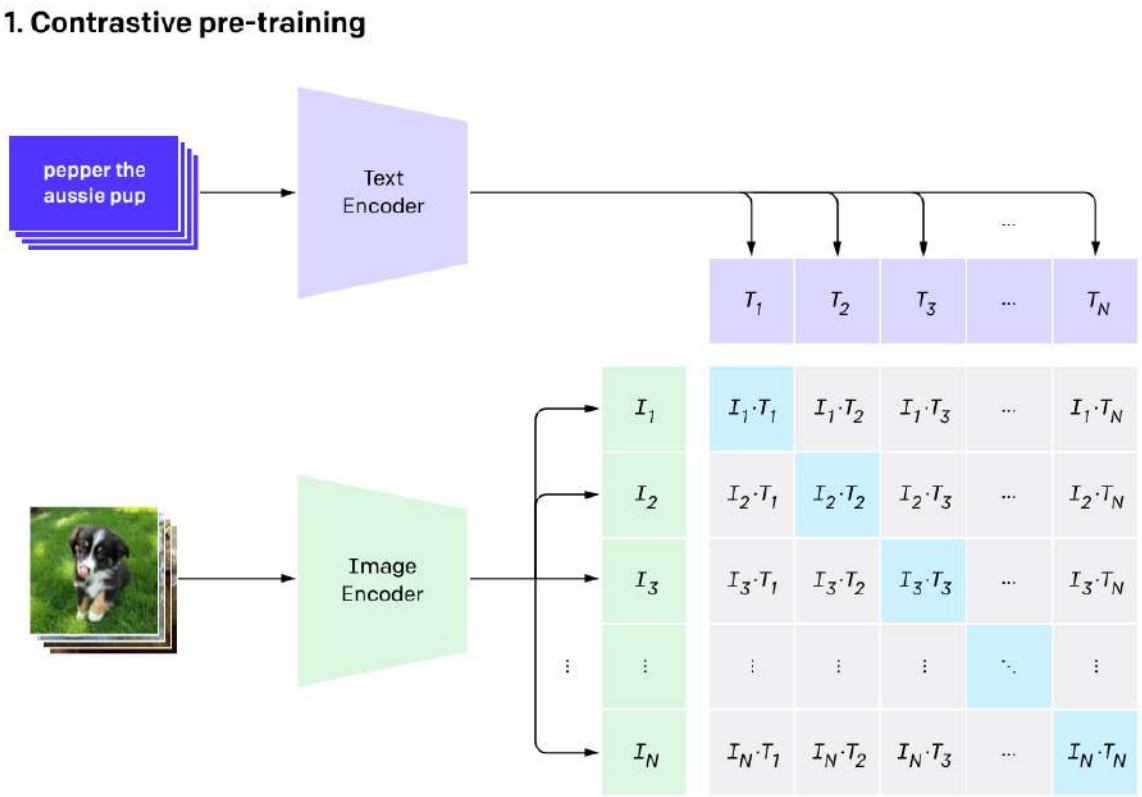
13

14

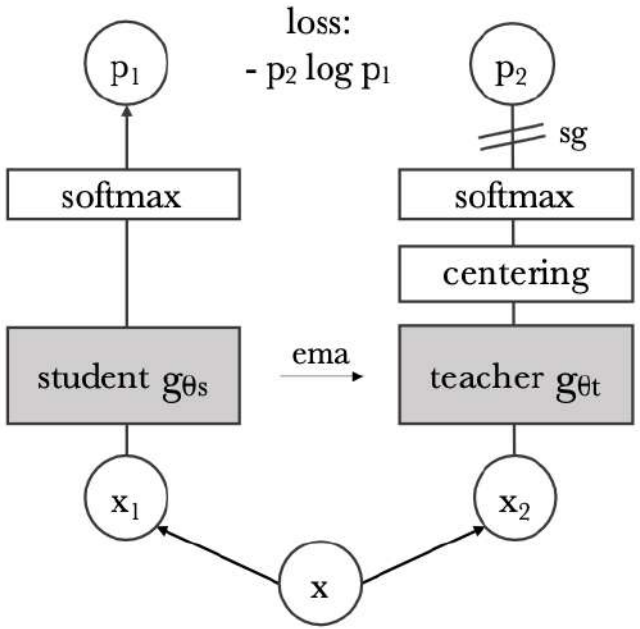
Foundation ViTs : Segment Anything Model (SAM)

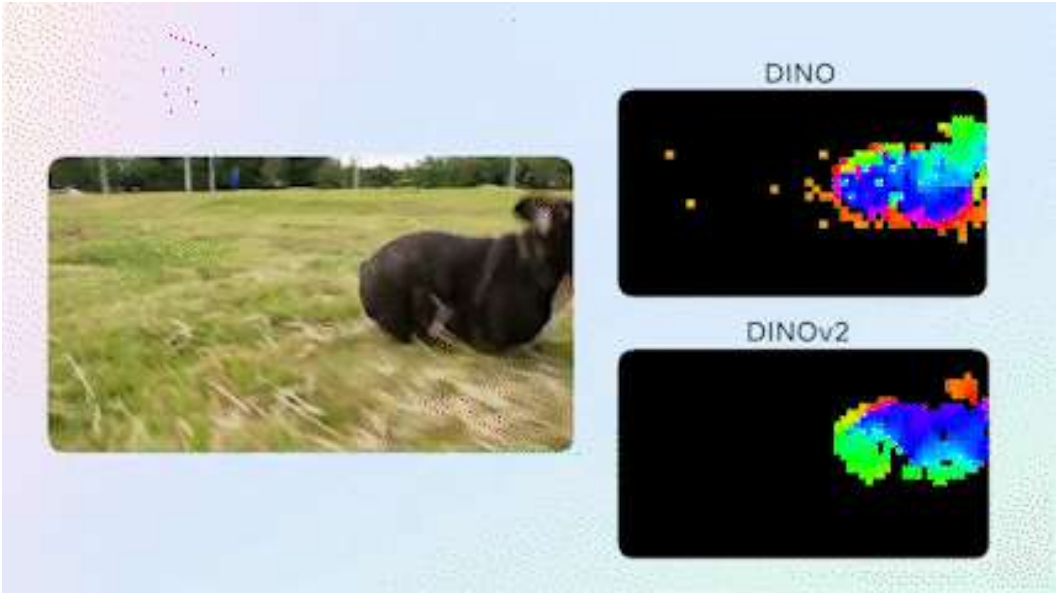



Open Vocab Models: CLIP (see next lecture)

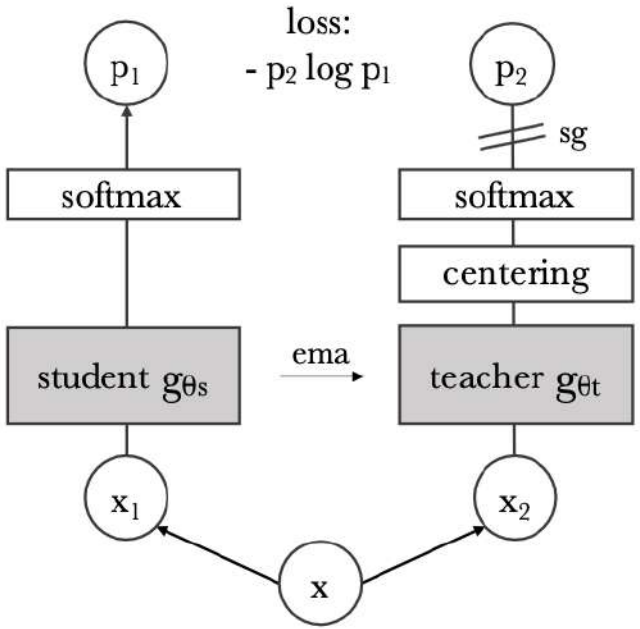


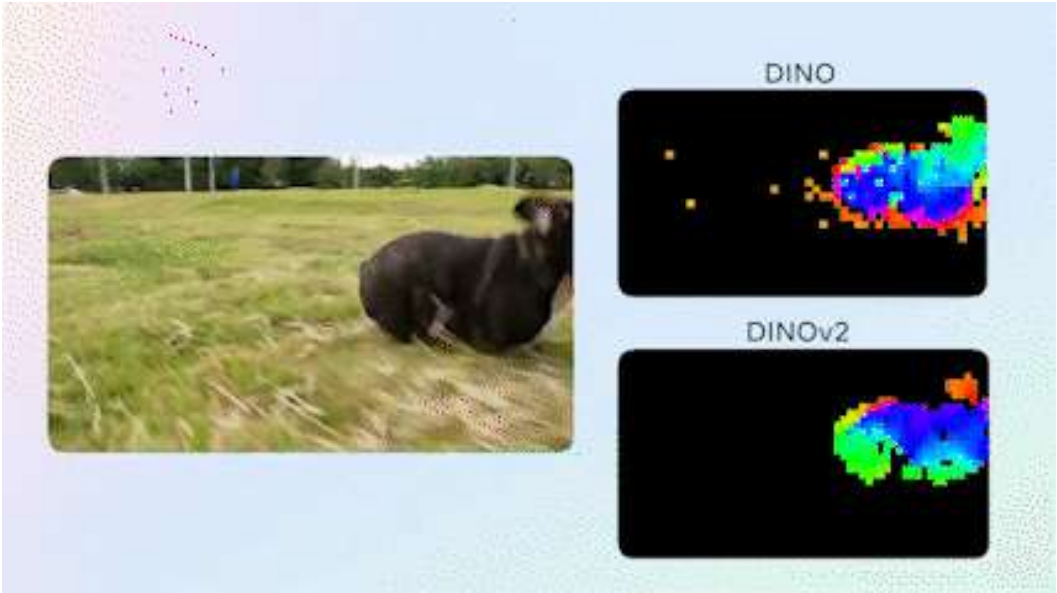
Self-supervised foundation features (plug-and-play embeddings): DINO



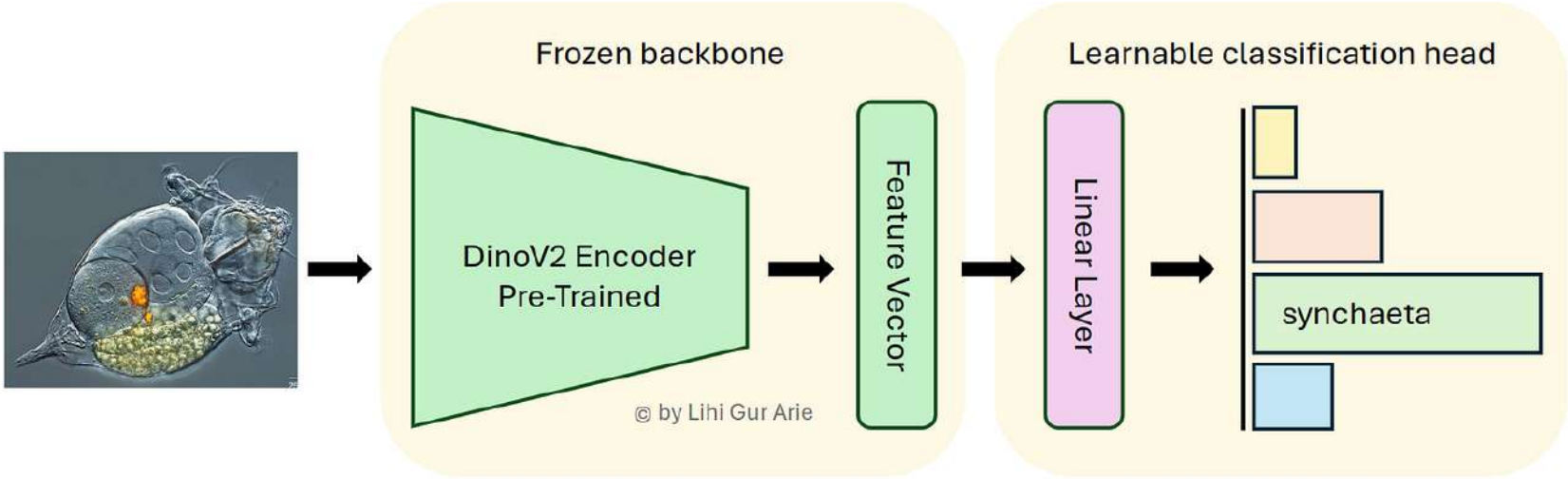


Self-supervised foundation features (plug-and-play embeddings): DINO

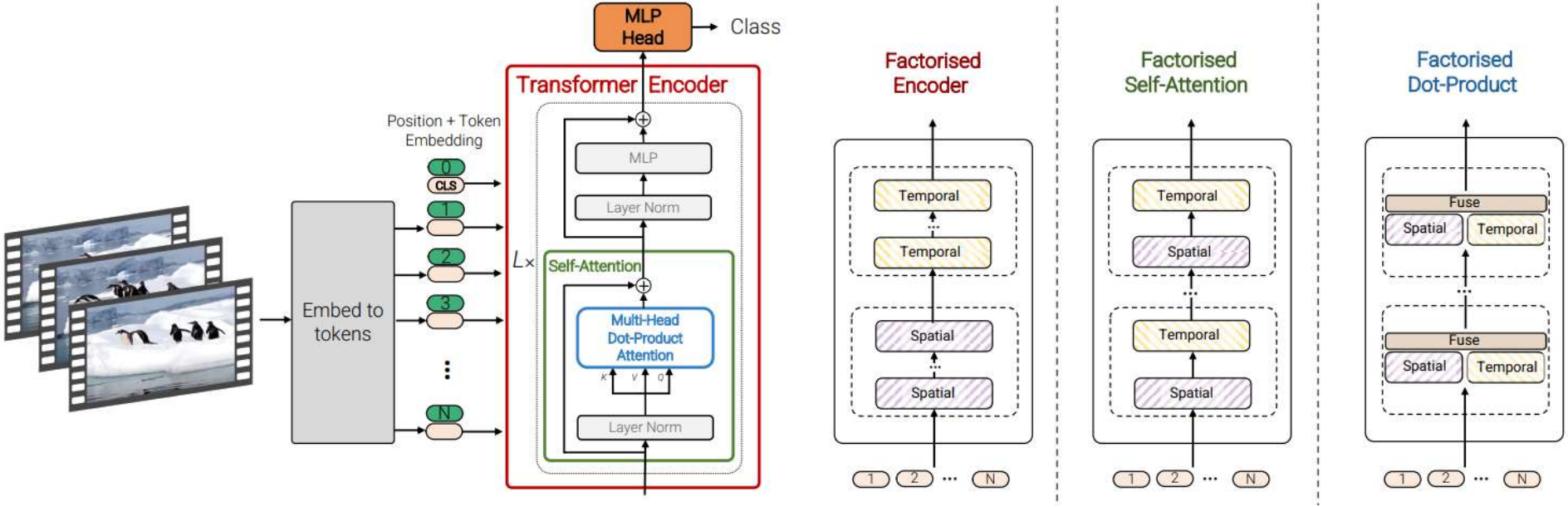




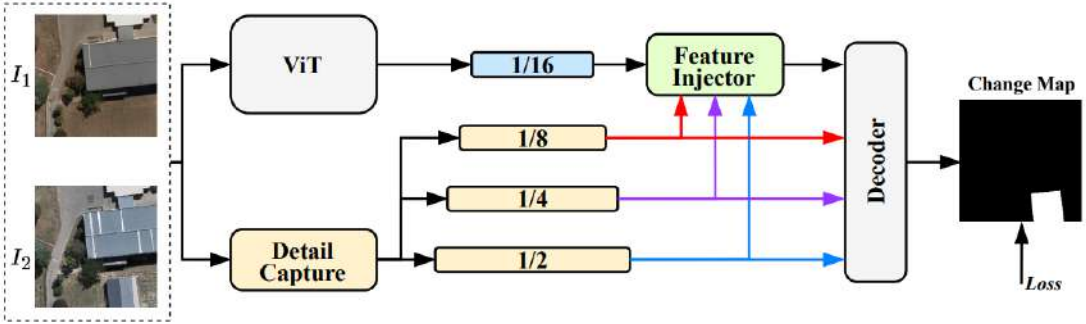
DINO Embeddings: Downstream uses

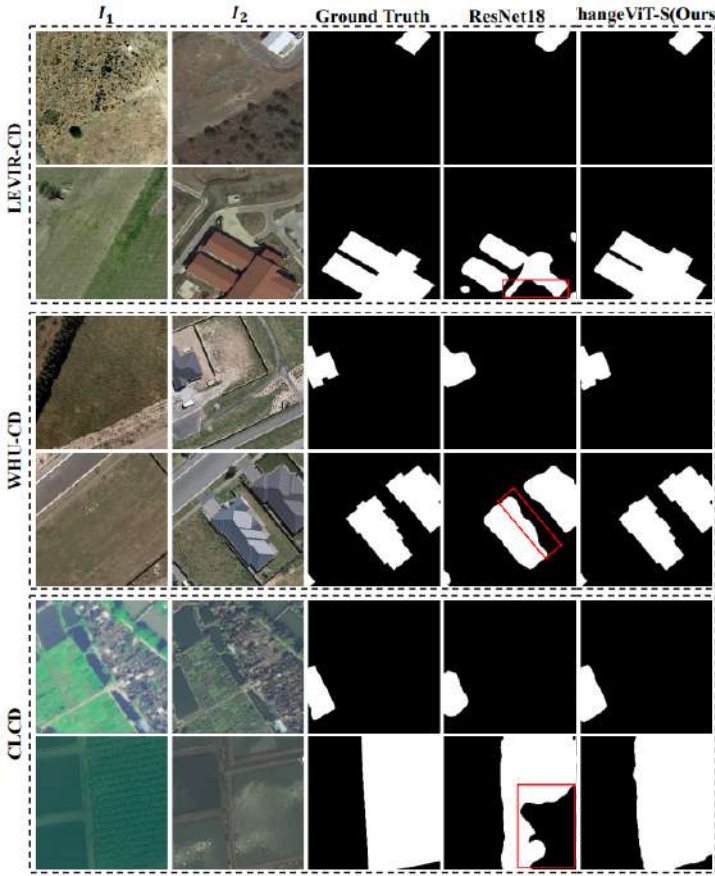


Video Understanding: ViViT



Remote sensing: ChangeViT





Autonomous driving perception (Maybe)